



UNIVERSIDADE FEDERAL DE SERGIPE

DEPARTAMENTO DE COMPUTAÇÃO

DISCIPLINA: ENGENHARIA DE SOFTWARE

Documento de Arquitetura de Software: Sistema de Gestão para Clínica Médica — MedClinic

Modelo 4+1 Visões | ISO/IEC/IEEE 42010:2011

Discentes:

Cézar Augusto Nascimento Dias

Gabriel Lima Dantas

Gabriel Rodrigues dos Santos

Lucas Andrey de Menezes Conrado

Orlando Gabriel Nascimento dos Santos

Rafael Rocha da Silva

Docente: Michel Dos Santos Soares

Maio de 2026

SUMÁRIO

1	Introdução	6
1.1	Propósito	6
1.2	Escopo	6
1.3	Avaliação da Descrição Arquitetural	6
1.4	Definições e Acrônimos	7
1.5	Documentos de Referência	8
2	Stakeholders e Concerns	8
2.1	Identificação de Stakeholders e seus Concerns	8
2.2	Descrição dos Concerns selecionados	9
2.2.1	Funcionalidade	10
2.2.2	Usabilidade e interface	10
2.2.3	Desempenho	10
2.2.4	Disponibilidade	10
2.2.5	Segurança e privacidade	10
2.2.6	Integridade dos dados	10
2.2.7	Rastreabilidade	10
2.2.8	Controle de estoque	11
2.2.9	Conformidade normativa	11
2.2.10	Modularidade	11
2.2.11	Modificabilidade	11
2.2.12	Testabilidade	11
2.2.13	Manutenibilidade	11
2.2.14	Organização do código	11
2.2.15	Cronograma e viabilidade	12
2.2.16	Separação de responsabilidades	12
2.2.17	Custo	12
2.2.18	Controle administrativo	12
2.2.19	Tomada de decisão	12
2.2.20	Qualidade dos artefatos	12
2.2.21	Rationale	12
2.2.22	Consistência entre visões	13
2.2.23	Deploy e manutenção operacional	13
2.2.24	Backup	13
2.2.25	Recuperação de falhas	13
2.2.26	Monitoramento	13
2.3	Matriz de Concerns e Stakeholders	13

2.4	Correspondência entre Concerns e Visões Arquiteturais	14
3	Definição dos Architecture Viewpoints	15
3.1	Viewpoint Lógico	15
3.2	Viewpoint de Processos	16
3.3	Viewpoint de Implementação	16
3.4	Viewpoint Físico	17
3.5	Viewpoint de Cenários	17
4	Representação da Arquitetura	17
5	Metas e Restrições Arquiteturais	18
6	Atributos de Qualidade	19
6.1	Desempenho	19
6.2	Escalabilidade	20
6.3	Modificabilidade	20
6.4	Segurança	20
6.5	Disponibilidade	21
6.6	Portabilidade	22
6.7	Testabilidade	22
6.8	Resumo de Tradeoffs	22
7	Padrões arquiteturais adotados	23
7.1	Cliente-servidor	23
7.2	Camadas	23
7.2.1	Camada de apresentação	24
7.2.2	Camada de negócio	24
7.2.3	Camada de dados	24
7.3	Estilo complementar: API REST	25
8	Modelo de Análise BCE (Boundary-Control-Entity)	25
8.1	Princípios de especificação BCE	26
8.2	Módulo de Gestão de Acesso	26
8.3	Módulo de Gestão de Cadastros	27
8.4	Módulo de Fluxo de Atendimento	27
8.5	Módulo de Prontuário Eletrônico	28
8.6	Módulo de Estoque de Materiais	28
8.7	Módulo de Estoque de Medicamentos e Dispensação	29
8.8	Correspondência BCE e Three-Tier	29

9	Visão Lógica (Logical View)	30
9.1	Propósito e preocupações arquiteturais	30
9.2	Pacotes e módulos principais	30
9.3	Estrutura em camadas dentro do backend	32
9.4	Principais relacionamentos entre módulos	32
9.5	Architecture Models da Visão Lógica (ISO 42010, Seção 5.6)	33
9.6	Inconsistências conhecidas (ISO 42010, Seção 5.7.1)	34
10	Visão de Processos (Process View)	34
10.1	Propósito e preocupações arquiteturais	34
10.2	Processos e <i>threads</i> principais	34
10.3	Comunicação entre componentes	35
10.4	Concorrência, integridade e falhas	36
10.5	Architecture Models desta Visão (ISO 42010, Seção 5.6)	36
10.6	Inconsistências conhecidas	37
11	Visão de Implementação (Development View)	37
11.1	Propósito e preocupações arquiteturais	37
11.2	Organização do repositório	37
11.3	Subcamadas do backend	38
11.4	Dependências tecnológicas	39
11.5	Pipeline de integração contínua	39
11.6	Architecture Models desta Visão (ISO 42010, Seção 5.6)	39
11.7	Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)	40
12	Visão Física (Deployment View)	40
12.1	Propósito e preocupações arquiteturais	40
12.2	Nós de infraestrutura	40
12.3	Fluxos de comunicação entre nós	41
12.4	Ambientes	42
12.5	Architecture Models desta Visão (ISO 42010, Seção 5.6)	42
12.6	Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)	43
13	Visão de Cenários (Scenario / Use Case View)	43
13.1	Propósito e preocupações arquiteturais	43
13.2	Cenário 1: médico registra consulta e gera prescrição	43
13.3	Cenário 2: recepcionista agenda consulta	44
13.4	Cenário 3: farmacêutico realiza dispensação com baixa de estoque	45
13.5	Architecture Models desta Visão (ISO 42010, Seção 5.6)	46
13.6	Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)	46

14 Decisões Arquiteturais — ISO/IEC/IEEE 42010	46
14.1 Rastreabilidade entre preocupações e decisões arquiteturais	46
14.2 Decisões arquiteturais registradas	47
14.2.1 DA-001 — Adoção do padrão Three-Tier com Cliente-Servidor	47
14.2.2 DA-002 — Autenticação stateless com JWT	48
14.2.3 DA-003 — API REST versionada como interface entre Frontend e Backend	49
14.2.4 DA-004 — Criptografia AES-256-GCM com chaves gerenciadas pelo AWS KMS	50
14.2.5 DA-005 — Upload direto de arquivos ao S3 via URL pré-assinada . . .	51
14.2.6 DA-006 — Single-AZ para RDS no escopo acadêmico	52
Referências	54

1 INTRODUÇÃO

1.1 Propósito

Este documento descreve a arquitetura de software do sistema MedClinic — Sistema de Gestão para Clínica Médica, desenvolvido no âmbito da disciplina de Engenharia de Software da Universidade Federal de Sergipe. O documento foi elaborado em conformidade com os requisitos normativos da ISO/IEC/IEEE 42010:2011 (Cláusula 5), utilizando o modelo de visões arquiteturais 4+1 (Kruchten, 1995) como architecture framework.

O objetivo principal é fornecer uma descrição estruturada e multi-perspectiva da arquitetura do sistema, servindo como base para as atividades de projeto detalhado, implementação, testes e manutenção.

1.2 Escopo

O documento abrange a arquitetura do MedClinic em sua totalidade, cobrindo os módulos de Gestão de Acesso, Gestão de Cadastros, Fluxo de Atendimento, Prontuário Eletrônico, Módulo Farmacêutico e Gestão Financeira, conforme especificado na SRS v4.1 (IEEE 830-1998).

1.3 Avaliação da Descrição Arquitetural

Conforme exigido pela ISO/IEC/IEEE 42010:2011, os resultados de avaliações da arquitetura e desta descrição arquitetural são registrados nesta seção.

Tabela 1 – Avaliação da descrição arquitetural

Critério de avaliação	Resultado	Observações
Conformidade com ISO/IEC/IEEE 42010:2011	Em conformidade	Stakeholders, concerns, viewpoints, views, models, rationale e correspondences presentes conforme Cláusula 5.
Adequação da arquitetura aos propósitos do sistema	Adequada	A Three-Tier + Cliente-Servidor satisfaz os requisitos funcionais e não funcionais da SRS v4.1 para o contexto acadêmico.
Viabilidade de construção e implantação	Viável	Equipe de 6 devs júnior, prazo de 14 semanas, infraestrutura AWS dentro do orçamento de R\$ 43.006,00.
Cobertura de concerns identificados	Completa	Todos os 11 concerns da Seção 2.2 são endereçados por ao menos uma visão (Seção 2.3).

Critério de avaliação	Resultado	Observações
Inconsistências conhecidas entre visões	Inconsistências conhecidas documentadas: divergências entre disponibilidade desejada e RDS Single-AZ; pendências de alinhamento entre módulos financeiros, casos de uso e mapeamento objeto-relacional.	Ver Seção 9.6, 10.6, 11.7, 12.6 e 13.6 — registro de inconsistências por visão.
Avaliação de riscos arquiteturais	Risco residual documentado	DA-006 registra o risco de disponibilidade associado ao RDS Single-AZ no escopo acadêmico.

1.4 Definições e Acrônimos

Tabela 2 – Definições e acrônimos

Termo / Acrônimo	Definição
AD	<i>Architecture Description</i> — descrição arquitetural conforme ISO/IEC/IEEE 42010:2011.
Concern	Interesse em um sistema relevante para um ou mais stakeholders.
Stakeholder	Indivíduo, equipe ou organização com interesse no sistema.
Visão	Expressão da arquitetura do sistema sob a perspectiva de concerns específicos.
Viewpoint	Convenções para construção, interpretação e uso de uma visão.
Model kind	Convenções para um tipo de modelo utilizado em uma visão.
Rationale	Justificativa registrada para uma decisão arquitetural.
BCE	Boundary-Control-Entity.
Three-Tier	Padrão arquitetural de três camadas: Apresentação, Negócio e Dados.
AWS	Amazon Web Services, provedor de nuvem utilizado para hospedagem.
EC2	Elastic Compute Cloud, servidor virtual AWS utilizado no backend.
RDS	Relational Database Service, banco de dados gerenciado AWS com PostgreSQL.
S3	Simple Storage Service, armazenamento de arquivos AWS.
REST	Representational State Transfer.
JWT	JSON Web Token.
RBAC	Role-Based Access Control.
ORM	Object-Relational Mapping.
CI/CD	Continuous Integration / Continuous Deployment.

1.5 Documentos de Referência

- KRUCHTEN, Philippe. Architectural Blueprints — The “4+1” View Model of Software Architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, nov. 1995.
- ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description. International Organization for Standardization, 2011.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. 3. ed. Addison-Wesley, 2012.
- Documento de Requisitos de Software — MedClinic SRS v4.1. Departamento de Computação — UFS, Maio de 2026.
- Descrição de Casos de Uso — MedClinic v3.0. Departamento de Computação — UFS, Maio de 2026.
- Proposta de Plano de Projeto — Sistema de Gestão para Clínica Médica. Departamento de Computação — UFS, Abril de 2026.
- Tabelas do Mapeamento Objeto-Relacional — MedClinic. Departamento de Computação — UFS, 2026.

2 STAKEHOLDERS E CONCERNS

Conforme a ISO/IEC/IEEE 42010:2011, uma descrição arquitetural deve identificar os stakeholders do sistema e os concerns relevantes para esses stakeholders. No MedClinic, os concerns foram selecionados de acordo com o contexto do projeto, considerando funcionalidades esperadas, propriedades de qualidade, restrições de custo e prazo, riscos, conformidade normativa e necessidades de manutenção.

Essa identificação constitui a base sobre a qual os viewpoints (Seção 3), as visões arquiteturais (Seções 9–13) e as decisões arquiteturais (Seção 14) são fundamentados.

2.1 Identificação de Stakeholders e seus Concerns

A tabela a seguir apresenta os principais stakeholders do MedClinic, seus papéis no sistema e os concerns arquiteturais associados a cada um. Os concerns não representam uma lista genérica de todos os exemplos possíveis da literatura, mas sim aqueles considerados relevantes para o domínio do sistema de gestão de clínica médica.

Tabela 3 – Stakeholders e concerns principais

Stakeholder	Papel no sistema	Concerns arquiteturais
Paciente	Usuário final externo ou beneficiário direto do sistema; consulta informações próprias e acompanha dados relacionados ao atendimento.	Funcionalidade; usabilidade e interface; segurança e privacidade; disponibilidade.
Recepcionista	Usuário interno responsável por cadastros, agendamentos e organização do fluxo inicial de atendimento.	Funcionalidade; usabilidade e interface; desempenho; disponibilidade; integridade dos dados.
Médico	Usuário interno responsável por consultas, registro de prontuário, prescrições e acompanhamento clínico do paciente.	Funcionalidade; desempenho; segurança e privacidade; integridade dos dados; conformidade normativa; rastreabilidade.
Enfermeiro	Usuário interno que consulta prontuários e acompanha o uso de materiais durante o atendimento.	Funcionalidade; usabilidade e interface; disponibilidade; segurança e privacidade; integridade dos dados; rastreabilidade; controle de estoque.
Farmacêutico	Usuário interno responsável pelo controle de medicamentos e pela dispensação vinculada ao prontuário.	Funcionalidade; desempenho; integridade dos dados; rastreabilidade; controle de estoque; disponibilidade.
Gestor / Administrador	Usuário interno avançado responsável por gerenciamento de usuários, estoques, relatórios e acompanhamento administrativo da clínica.	Funcionalidade; disponibilidade; segurança e privacidade; custo; controle administrativo; tomada de decisão.
Equipe de Desenvolvimento	Equipe responsável por projetar, implementar, testar e manter o sistema no contexto acadêmico.	Modularidade; modificabilidade; testabilidade; manutenibilidade; organização do código; cronograma e viabilidade; separação de responsabilidades.
Docente Orientador	Stakeholder avaliador; verifica a aderência do projeto aos conceitos, padrões e artefatos de Engenharia de Software.	Conformidade normativa; qualidade dos artefatos; rastreabilidade; rationale; consistência entre visões.
Administrador de TI da Clínica	Responsável por operar, implantar e manter o sistema após a entrega acadêmica.	Deploy e manutenção operacional; disponibilidade; segurança e privacidade; backup; recuperação de falhas; monitoramento; custo.

2.2 Descrição dos Concerns selecionados

Os concerns abaixo representam os interesses arquiteturalmente relevantes para o Med-Clinic. Cada concern é descrito no contexto específico do projeto, evitando a utilização de

termos genéricos sem relação direta com o sistema.

2.2.1 Funcionalidade

A funcionalidade refere-se à capacidade do sistema de atender às funções essenciais da clínica. No MedClinic, esse concern envolve autenticação de usuários, cadastro de pacientes, agendamento de consultas, consulta e atualização de prontuário, controle de materiais, controle de medicamentos, dispensação de medicamentos e geração de informações administrativas.

2.2.2 Usabilidade e interface

A usabilidade e interface referem-se à facilidade de uso das telas e dos fluxos do sistema. No MedClinic, esse concern envolve clareza visual, organização dos formulários, redução de passos em tarefas frequentes e facilidade de operação por usuários internos da clínica, como recepcionistas, médicos, enfermeiros e farmacêuticos.

2.2.3 Desempenho

O desempenho refere-se ao tempo de resposta do sistema durante operações rotineiras. No MedClinic, esse concern é relevante principalmente em ações como busca de pacientes, agendamento de consultas, consulta de prontuário, verificação de estoque e dispensação de medicamentos.

2.2.4 Disponibilidade

A disponibilidade refere-se à capacidade do sistema de permanecer acessível durante o período de funcionamento da clínica. No MedClinic, a indisponibilidade do sistema pode prejudicar atendimentos, agendamentos, consultas a prontuários e atividades de controle de estoque.

2.2.5 Segurança e privacidade

A segurança e privacidade referem-se à proteção dos dados sensíveis de pacientes e profissionais. No MedClinic, esse concern envolve autenticação, autorização por perfil, criptografia, controle de acesso e prevenção de acessos indevidos às informações clínicas e administrativas.

2.2.6 Integridade dos dados

A integridade dos dados refere-se à garantia de que as informações armazenadas e manipuladas pelo sistema permaneçam corretas e consistentes. No MedClinic, esse concern envolve dados de pacientes, agendamentos, consultas, prontuários, materiais, medicamentos, consumos e dispensações.

2.2.7 Rastreabilidade

A rastreabilidade refere-se à capacidade de identificar a origem, o responsável e o histórico de determinadas operações realizadas no sistema. No MedClinic, esse concern é importante

para acompanhar acessos ao prontuário, uso de materiais, dispensação de medicamentos e alterações em informações clínicas.

2.2.8 Controle de estoque

O controle de estoque refere-se ao acompanhamento das quantidades, lotes, validades e níveis mínimos de materiais e medicamentos. No MedClinic, esse concern está relacionado à disponibilidade de insumos para atendimento e à prevenção de falta de materiais ou medicamentos necessários à operação da clínica.

2.2.9 Conformidade normativa

A conformidade normativa refere-se ao atendimento a normas e boas práticas relacionadas à proteção de dados e ao tratamento de informações clínicas. No MedClinic, esse concern está associado principalmente à LGPD, às diretrizes aplicáveis ao uso de prontuários eletrônicos e à organização da documentação arquitetural segundo a ISO/IEC/IEEE 42010:2011.

2.2.10 Modularidade

A modularidade refere-se à organização do sistema em partes com responsabilidades bem definidas. No MedClinic, esse concern é representado pela divisão em módulos como Acesso, Cadastros, Atendimento, Prontuário, Estoque de Materiais e Estoque de Medicamentos.

2.2.11 Modificabilidade

A modificabilidade refere-se à facilidade de alterar ou evoluir funcionalidades sem causar impacto excessivo em outras partes da aplicação. No MedClinic, esse concern é importante porque o sistema é desenvolvido em contexto acadêmico e pode sofrer alterações de requisitos, regras de negócio e escopo ao longo do projeto.

2.2.12 Testabilidade

A testabilidade refere-se à facilidade de testar regras de negócio, endpoints, integrações e módulos do sistema. No MedClinic, esse concern permite validar funcionalidades como autenticação, agendamento, acesso ao prontuário, controle de estoque e dispensação de medicamentos, reduzindo a ocorrência de defeitos.

2.2.13 Manutenibilidade

A manutenibilidade refere-se à facilidade de corrigir defeitos, ajustar regras de negócio e manter o sistema compreensível ao longo do tempo. No MedClinic, esse concern é favorecido pela separação em camadas, pela organização modular e pela divisão clara de responsabilidades.

2.2.14 Organização do código

A organização do código refere-se à forma como os arquivos, pacotes, módulos e camadas da aplicação são estruturados. No MedClinic, esse concern é relevante para facilitar o

trabalho da equipe de desenvolvimento, a localização de funcionalidades e a evolução do sistema.

2.2.15 Cronograma e viabilidade

O cronograma e a viabilidade referem-se à possibilidade de construir, testar e documentar o sistema dentro do prazo acadêmico e com a equipe disponível. No MedClinic, esse concern influencia a adoção de uma arquitetura Three-Tier modular, evitando alternativas mais complexas para o contexto do projeto.

2.2.16 Separação de responsabilidades

A separação de responsabilidades refere-se à divisão clara entre as partes do sistema, evitando que uma mesma camada ou módulo concentre funções excessivas. No MedClinic, esse concern aparece na separação entre apresentação, regras de negócio e persistência, bem como na divisão dos módulos funcionais.

2.2.17 Custo

O custo refere-se à adequação das escolhas arquiteturais ao orçamento previsto para o projeto. No MedClinic, esse concern influencia decisões relacionadas à infraestrutura em nuvem, ao uso de serviços AWS e à escolha de uma arquitetura compatível com o porte acadêmico do sistema.

2.2.18 Controle administrativo

O controle administrativo refere-se à capacidade de acompanhar e gerenciar aspectos internos da clínica, como usuários, estoques, relatórios e informações de apoio à gestão. No MedClinic, esse concern está principalmente associado ao perfil Gestor / Administrador.

2.2.19 Tomada de decisão

A tomada de decisão refere-se ao uso das informações do sistema para apoiar decisões gerenciais. No MedClinic, esse concern envolve o acesso a relatórios, métricas de produtividade, dados de estoque e informações administrativas da clínica.

2.2.20 Qualidade dos artefatos

A qualidade dos artefatos refere-se à clareza, consistência e completude da documentação produzida no projeto. No MedClinic, esse concern está associado à avaliação acadêmica dos documentos de requisitos, casos de uso, mapeamento objeto-relacional, arquitetura e decisões arquiteturais.

2.2.21 Rationale

O rationale refere-se ao registro das justificativas das decisões arquiteturais. No MedClinic, esse concern envolve documentar por que determinadas alternativas foram escolhidas,

quais opções foram descartadas, quais trade-offs foram considerados e quais consequências arquiteturais foram assumidas.

2.2.22 Consistência entre visões

A consistência entre visões refere-se ao alinhamento entre as diferentes visões arquiteturais do documento. No MedClinic, esse concern exige que a Visão Lógica, a Visão de Processos, a Visão de Implementação, a Visão Física e a Visão de Cenários utilizem conceitos compatíveis entre si.

2.2.23 Deploy e manutenção operacional

O deploy e a manutenção operacional referem-se à facilidade de implantar, configurar, monitorar, restaurar e manter o sistema após a entrega. No MedClinic, esse concern envolve publicação da aplicação, configuração da infraestrutura, backup, rollback, monitoramento e recuperação de falhas.

2.2.24 Backup

O backup refere-se à cópia periódica dos dados necessários para recuperação do sistema em caso de falhas. No MedClinic, esse concern é importante para reduzir o risco de perda de prontuários, agendamentos, dados cadastrais e registros de estoque.

2.2.25 Recuperação de falhas

A recuperação de falhas refere-se à capacidade de restaurar o funcionamento do sistema após indisponibilidades, erros de infraestrutura ou problemas operacionais. No MedClinic, esse concern está relacionado à continuidade dos atendimentos e à preservação dos dados.

2.2.26 Monitoramento

O monitoramento refere-se ao acompanhamento do funcionamento da aplicação e da infraestrutura. No MedClinic, esse concern permite identificar indisponibilidades, falhas persistentes, uso excessivo de recursos e problemas que possam comprometer a operação da clínica.

2.3 Matriz de Concerns e Stakeholders

A tabela abaixo evidencia que um mesmo concern pode ser compartilhado por múltiplos stakeholders. Essa matriz é usada para fundamentar as decisões arquiteturais da Seção 14, mostrando quais stakeholders são afetados por cada preocupação arquitetural.

Tabela 4 – Matriz de concerns e stakeholders

Concern	Stakeholders associados
Funcionalidade	Paciente, Recepcionista, Médico, Enfermeiro, Farmacêutico, Gestor.

Concern	Stakeholders associados
Usabilidade e interface	Paciente, Recepcionista, Médico, Enfermeiro, Farmacêutico.
Desempenho	Recepcionista, Médico, Enfermeiro, Farmacêutico, Paciente.
Disponibilidade	Paciente, Recepcionista, Médico, Enfermeiro, Farmacêutico, Gestor, Administrador de TI.
Segurança e privacidade	Paciente, Médico, Gestor, Administrador de TI, Docente Orientador.
Integridade dos dados	Médico, Enfermeiro, Farmacêutico, Gestor, Administrador de TI.
Conformidade normativa	Paciente, Médico, Gestor, Docente Orientador, Administrador de TI.
Modularidade	Equipe de Desenvolvimento, Docente Orientador, Administrador de TI.
Modificabilidade	Equipe de Desenvolvimento, Docente Orientador, Administrador de TI.
Testabilidade	Equipe de Desenvolvimento, Docente Orientador.
Custo	Gestor, Equipe de Desenvolvimento, Administrador de TI.
Cronograma e viabilidade	Equipe de Desenvolvimento, Docente Orientador, Gestor.
Deploy e manutenção operacional	Administrador de TI, Equipe de Desenvolvimento, Gestor.

2.4 Correspondência entre Concerns e Visões Arquiteturais

Cada visão arquitetural está relacionada a um ou mais concerns. A tabela abaixo explicita essa correspondência para o MedClinic, mostrando como as cinco visões do modelo 4+1 endereçam coletivamente os concerns identificados.

Tabela 5 – Correspondência entre concerns e visões

Visão	Concerns endereçados	Stakeholders primários
Lógica (Seção 9)	Funcionalidade; modularidade; modificabilidade; integridade dos dados; organização estrutural; separação de responsabilidades.	Equipe de Desenvolvimento, Docente Orientador, Médico, Farmacêutico.
Processos (Seção 10)	Desempenho; disponibilidade; integridade dos dados; concorrência; tolerância a falhas; comportamento em tempo de execução.	Médico, Farmacêutico, Recepcionista, Administrador de TI.
Implementação (Seção 11)	Modularidade; modificabilidade; testabilidade; organização do código; reuso; cronograma e viabilidade.	Equipe de Desenvolvimento, Docente Orientador, Gestor.
Física (Seção 12)	Disponibilidade; segurança e privacidade; custo; deploy e manutenção operacional; backup; recuperação de falhas.	Administrador de TI, Gestor, Equipe de Desenvolvimento.

Visão	Concerns endereçados	Stakeholders primários
Cenários (Seção 13)	Funcionalidade; usabilidade e interface; integridade dos dados; validação da arquitetura; consistência entre visões; ponto de partida para testes.	Todos os stakeholders.

3 DEFINIÇÃO DOS ARCHITECTURE VIEWPOINTS

A Cláusula 5.4 da ISO/IEC/IEEE 42010:2011 exige que cada architecture description inclua a definição de cada viewpoint utilizado, especificando: (a) os concerns que enquadra; (b) os stakeholders típicos; (c) os model kinds utilizados; (d) as linguagens, notações e convenções de cada model kind; (e) referências às fontes. Esta seção cumpre esse requisito para os cinco viewpoints do modelo 4+1.

A adoção do modelo 4+1 como framework arquitetural foi justificada por sua adequação aos concerns do MedClinic e por sua capacidade de organizar a arquitetura em múltiplas visões complementares: Lógica, Processos, Implementação, Física e Cenários. O MedClinic é um sistema de informação web construído com camadas bem definidas, banco de dados relacional e acesso via navegador. Para essa realidade, o 4+1 atende às necessidades do projeto, pois mapeia a estrutura funcional na Visão Lógica, o comportamento de execução na Visão de Processos, a organização do código na Visão de Implementação, a infraestrutura na Visão Física e a validação do sistema por meio da Visão de Cenários.

Foram descartadas alternativas como RM-ODP e TOGAF, pois apresentam escopo mais amplo e maior complexidade documental do que o necessário para o porte do sistema e para o contexto acadêmico do projeto.

3.1 Viewpoint Lógico

Tabela 6 – Viewpoint lógico

Campo (ISO 42010, Cl. 7)	Especificação
Concerns enquadrados	Estrutura funcional, separação de responsabilidades, modularidade, complexidade, modificabilidade e evoluibilidade do sistema.
Stakeholders típicos	Equipe de Desenvolvimento, Arquiteto, Docente Orientador.
Model kinds	MK-L1: Diagrama de Pacotes e Módulos; MK-L2: Tabela BCE por módulo; MK-L3: Relacionamentos entre módulos.
Notações e convenções	MK-L1: notação textual estruturada em tabela (UML Package Diagram seria o equivalente gráfico); MK-L2: tabela colorida por estereótipo BCE conforme Parte 13 dos slides do professor (Diagrama de Classes UML com estereótipos).

Campo	Especificação
Correspondências	Os módulos (MK-L1) correspondem às subcamadas de implementação (Viewpoint de Implementação, Seção 3.3). As classes Entity (MK-L2) correspondem às entidades persistidas no Viewpoint Físico (Seção 3.4).
Fontes	ISO/IEC/IEEE 42010:2011; Kruchten (1995); slides de BCE.

3.2 Viewpoint de Processos

Tabela 7 – Viewpoint de processos

Campo (ISO 42010, Cl. 7)	Especificação
Concerns enquadrados	Concorrência, comunicação entre processos, integridade transacional, tolerância a falhas, desempenho em tempo de execução e disponibilidade.
Stakeholders típicos	Arquiteto, Equipe de Desenvolvimento Backend, Médico, Farmacêutico, Recepcionista (impactados por desempenho e disponibilidade).
Model kinds	MK-P1: Processos e threads; MK-P2: Fluxo de requisição HTTP; MK-P3: Transações críticas.
Notações e convenções	Descrição sequencial textual equivalente a diagramas UML de sequência e atividades.
Correspondências	Os processos identificados (MK-P1) são executados nos nós físicos descritos no Viewpoint Físico. Os serviços invocados correspondem aos módulos do Viewpoint Lógico.
Fontes	Kruchten (1995); ISO/IEC/IEEE 42010:2011; Bass et al. (2012) — Cap. Performance e Availability.

3.3 Viewpoint de Implementação

Tabela 8 – Viewpoint de implementação

Campo (ISO 42010, Cl. 7)	Especificação
Concerns enquadrados	Organização do código-fonte, modificabilidade, testabilidade, divisão de tarefas entre equipe, reuso, facilidade de deploy e integração contínua.
Stakeholders típicos	Equipe de Desenvolvimento, Gestor de Projeto, Docente Orientador.
Model kinds	MK-D1: Estrutura de repositório; MK-D2: Dependências tecnológicas; MK-D3: Pipeline CI/CD.
Notações e convenções	Estrutura de diretórios e tabelas, equivalentes textuais de diagramas de componentes UML.
Correspondências	Artefatos de código implementam módulos lógicos e são implantados nos nós físicos.

Campo	Especificação
Fontes	ISO/IEC/IEEE 42010:2011; Kruchten (1995); slides de implementação/deployability.

3.4 Viewpoint Físico

Tabela 9 – Viewpoint físico

Campo (ISO 42010, Cl. 7)	Especificação
Concerns enquadrados	Mapeamento de componentes em nós físicos, disponibilidade de infraestrutura, segurança de rede, custo de operação e configurações por ambiente.
Stakeholders típicos	Administrador de TI da Clínica, Gestor, Arquiteto, Equipe de Desenvolvimento.
Model kinds	MK-F1: Nós de infraestrutura; MK-F2: Fluxos de comunicação; MK-F3: Ambientes.
Notações e convenções	Tabelas estruturadas equivalentes a diagramas UML de implantação.
Correspondências	Nós hospedam artefatos do viewpoint de implementação; processos executam sobre os nós físicos.
Fontes	ISO/IEC/IEEE 42010:2011; Kruchten (1995).

3.5 Viewpoint de Cenários

Tabela 10 – Viewpoint de cenários

Campo (ISO 42010, Cl. 7)	Especificação
Concerns enquadrados	Integração e consistência entre as quatro visões anteriores; validação de casos de uso arquiteturalmente significativos.
Stakeholders típicos	Todos os stakeholders.
Model kinds	MK-C1: Cenários arquiteturalmente significativos.
Notações e convenções	Passos numerados em linguagem natural, equivalentes a diagramas UML de sequência.
Correspondências	Cada passo referencia módulos lógicos, processos, artefatos de implementação e nós físicos.
Fontes	ISO/IEC/IEEE 42010:2011; Kruchten (1995); casos de uso MedClinic.

4 REPRESENTAÇÃO DA ARQUITETURA

A arquitetura do MedClinic é descrita por meio do modelo de visões múltiplas 4+1, proposto por Kruchten (1995). Esse modelo organiza a descrição arquitetural em cinco visões complementares, cada uma endereçando os concerns de diferentes grupos de stakeholders. Con-

forme estabelecido pela ISO/IEC/IEEE 42010:2011, cada visão é associada a um viewpoint específico que define as convenções e notações utilizadas para expressá-la.

Tabela 11 – Visões arquiteturais do MedClinic

Visão	Concern central	Stakeholders primários	Notação principal
Lógica	Estrutura de classes, pacotes e responsabilidades funcionais.	Desenvolvedores, Arquiteto.	Classes e pacotes UML.
Processos	Concorrência, comunicação entre processos e tolerância a falhas.	Arquiteto, backend.	Diagrama de Sequência / Atividades UML.
Implementação	Organização de módulos, camadas e repositório de código.	Desenvolvedores, gestor.	Diagrama de Componentes UML.
Física	Mapeamento de componentes em nós físicos e nuvem.	Arquiteto, TI, Gestor.	Diagrama de Implantação UML.
Cenários	Integração das visões por casos de uso arquiteturalmente significativos.	Todos.	Casos de uso + Sequência UML.

As visões não são independentes entre si: a Visão Lógica define abstrações instanciadas na Visão de Processos; a Visão de Implementação organiza os artefatos que realizam a estrutura lógica; a Visão Física mapeia os componentes de implementação em infraestrutura real; e a Visão de Cenários valida o funcionamento integrado das demais visões.

5 METAS E RESTRIÇÕES ARQUITETURAIS

As metas e restrições a seguir definem o espaço de soluções dentro do qual a arquitetura do MedClinic foi projetada. Cada item representa um concern de alto nível identificado a partir dos requisitos não funcionais (SRS v4.1, Seção 3.2) e das restrições gerais do projeto (SRS v4.1, Seção 2.4).

Tabela 12 – Metas e restrições arquiteturais

Meta / Restrição	Tipo	Impacto arquitetural
Entrega como aplicação web (SaaS), sem instalação local	Restrição	Define Cliente-Servidor; exige frontend web e backend em nuvem.
Hospedagem exclusiva na AWS (EC2, RDS, S3)	Restrição	Condiciona a visão física ao ecossistema AWS e limita escolhas de infraestrutura.
Banco de dados relacional PostgreSQL v16.x	Restrição	Define mecanismo de persistência e exige ORM compatível.
Toda comunicação via HTTPS com TLS 1.3	Restrição	Impõe segurança nas interfaces de comunicação.

Meta / Restrição	Tipo	Impacto arquitetural
Suporte a 50 usuários simultâneos	Meta de desempenho	Orienta dimensionamento da instância EC2 e estratégia de conexões ao banco.
Controle de acesso granular por perfil (RBAC)	Meta de segurança	Requer autenticação JWT e autorização RBAC em todas as rotas da API.
Conformidade com LGPD e Resolução CFM nº 1.821/2007	Restrição normativa	Exige criptografia, exclusão lógica, logs de auditoria e controle de acesso.
Disponibilidade de 95% ao mês (Single-AZ)	Meta de disponibilidade	Define backup automático, restauração e monitoramento.
Equipe de 6 desenvolvedores júnior, 10h semanais	Restrição de equipe	Favorece arquitetura modular com responsabilidades bem separadas.
Prazo acadêmico até julho de 2026	Restrição de prazo	Limita a complexidade da solução; favorece Three-Tier modular em vez de microserviços.
Interface em Português Brasileiro (pt-BR), WCAG 2.1 AA	Meta de usabilidade	Orienta padrões de interface, acessibilidade e validações no frontend.

6 ATRIBUTOS DE QUALIDADE

Os atributos de qualidade representam os requisitos não funcionais que a arquitetura do sistema precisa atender. Como apontam Bass et al. (2012), uma boa arquitetura nasce de decisões intencionais que buscam equilibrar características que muitas vezes competem entre si. Na prática, isso significa que frequentemente é necessário sacrificar um aspecto para priorizar outro. A seguir, detalhamos os principais atributos de qualidade do MedClinic, as escolhas de projeto feitas para garanti-los e as concessões geradas por essas decisões.

6.1 Desempenho

O sistema precisa responder a 95% das requisições GET em até dois segundos, suportando até 50 usuários simultâneos, conforme especificado nos requisitos RNF-001 e RNF-002 do documento SRS v4.1. A escolha pela arquitetura em camadas beneficia o desempenho ao concentrar a lógica de negócio no backend, o que evita sobrecarregar o cliente com processamentos desnecessários.

Na camada de dados, a utilização do PostgreSQL com índices aplicados nas colunas de busca mais frequente (como `paciente.cpf`, `consulta.data-hora` e `estoque.medicamento-id`) e a separação das operações de leitura e escrita reduzem o tempo de espera nas ações mais rotineiras do sistema.

A métrica adotada para avaliar o desempenho é o tempo de resposta. A meta é manter 95% dos acessos abaixo de dois segundos e garantir que nenhum ultrapasse cinco segundos em situações normais de uso. Na interface, a primeira renderização de conteúdo na tela (*First Contentful Paint*) foi limitada a três segundos em conexões de 10 Mbps, em atendimento ao requisito RNF-003.

Trade-off: a adoção de múltiplas camadas introduz overhead em cada chamada entre camadas (apresentação - negócio - dados), impactando ligeiramente o desempenho em relação a uma arquitetura monolítica sem separação. Esse custo é considerado aceitável dada a melhoria em manutenibilidade e modificabilidade.

6.2 Escalabilidade

O sistema foi projetado para suportar o crescimento progressivo no volume de dados e no número de usuários simultâneos sem demandar uma reestruturação de sua base arquitetural. A segregação clara entre as camadas de *frontend* (estático), *backend* (hospedado em instâncias AWS EC2) e banco de dados (gerenciado via AWS RDS) permite o redimensionamento independente de cada componente conforme a demanda de carga. O caráter *stateless* do *backend* - assegurado pela adoção de *tokens* JWT em substituição ao armazenamento de sessões no servidor - viabiliza a escalabilidade horizontal. Isso facilita a inclusão de novas instâncias por trás de um balanceador de carga (*load balancer*) sempre que necessário, sem exigir qualquer alteração no código-fonte da aplicação.

Trade-off: Atualmente, o uso de uma instância RDS em modo *Single-AZ* (*db.t3.micro*) estabelece um gargalo para a escalabilidade vertical em cenários de expansão acentuada. Contudo, essa configuração é plenamente aceitável para o atual escopo acadêmico do projeto, estando a migração para um ambiente *Multi-AZ* formalmente documentada como uma decisão arquitetural futura (DA-006).

6.3 Modificabilidade

A modificabilidade representa um atributo central para o ecossistema do MedClinic, visto que o sistema se encontra em uma etapa de validação acadêmica na qual os requisitos evoluem com frequência. A arquitetura estruturada em camadas promove uma alta coesão interna e um baixo acoplamento entre os módulos, o que confere maior previsibilidade e isolamento a eventuais alterações do código. A separação entre as interfaces de usuário e a lógica de negócios é consolidada por meio de uma API REST devidamente versionada (*/api/v1/*), garantindo que evoluções em uma camada não exijam modificações simultâneas na outra. Adicionalmente, o emprego de uma ferramenta de ORM (*Object-Relational Mapping*) minimiza os reflexos de atualizações estruturais do banco de dados sobre as regras de negócio.

Trade-off: A rigidez na comunicação entre as camadas adjacentes pode, eventualmente, introduzir códigos de passagem direta (*pass-through*). Esse comportamento ocorre quando dados brutos precisam ser expostos diretamente ao *frontend*, acarretando uma leve penalidade de desempenho em operações mais simples e diretas.

6.4 Segurança

Considerando o tratamento de informações médicas e os estritos parâmetros regulatórios da Lei Geral de Proteção de Dados (LGPD), a segurança foi tratada como um requisito crítico

do projeto. A arquitetura mitiga vulnerabilidades por meio de uma abordagem de proteção em profundidade distribuída em múltiplas frentes:

- a) Criptografia de tráfego de rede utilizando o protocolo TLS 1.3 em todos os canais de comunicação;
- b) Autenticação baseada em JWT com tempo de expiração definido em 30 minutos e *refresh tokens* válidos por até 8 horas;
- c) Controle de acesso baseado em papéis (RBAC - *Role-Based Access Control*) embutido nos *middlewares* do *backend*, validando as permissões do usuário antes de processar qualquer requisição;
- d) Proteção de dados sensíveis em repouso por meio do algoritmo de criptografia AES-256-GCM, com chaves gerenciadas através do AWS KMS;
- e) Cifragem de credenciais de acesso utilizando funções de *hashing* robustas como *bcrypt* (fator de custo 12) ou Argon2id;
- f) Gerenciamento centralizado de variáveis de ambiente e credenciais críticas por meio do AWS Secrets Manager, impedindo a exposição de segredos no código-fonte.

Trade-off: A imposição de checagens de permissão a cada rota da API, combinada às operações de cifragem e decifragem de registros sensíveis, adiciona uma latência residual ao tempo de resposta das requisições. Este impacto na performance é assumido como obrigatório, dada a sensibilidade e a confidencialidade inerentes aos dados de saúde do sistema.

6.5 Disponibilidade

A meta estipulada para a disponibilidade interna do sistema é de 95% mensais, índice alinhado ao Acordo de Nível de Serviço (SLA) real da instância AWS RDS *Single-AZ (db.t3.micro)* adotada. Para mitigar riscos de perda de dados, a infraestrutura conta com rotinas automáticas de *backup* diário com retenção de 7 dias (RNF-011) e tempo estimado de recuperação (RTO) de até 15 minutos (RNF-012). O modelo *stateless* do *backend* permite reiniciar a instância EC2 rapidamente, sem derrubar ou invalidar as sessões ativas dos usuários. O monitoramento contínuo é realizado pelo AWS CloudWatch, configurado para disparar alertas caso o sistema apresente qualquer instabilidade persistente por mais de 5 minutos.

Trade-off: A escolha estratégica por uma topologia *Single-AZ* com foco em redução de custos operacionais implica que o sistema ficará temporariamente indisponível durante janelas de manutenção programada da AWS ou em falhas físicas na zona de disponibilidade. Para um cenário de produção comercial real, recomenda-se a transição para um arranjo *Multi-AZ*, elevando o SLA para 99,95

6.6 Portabilidade

O MedClinic pode ser acessado de forma ubíqua a partir de qualquer dispositivo equipado com um navegador web homologado, tais como Google Chrome (v120+), Mozilla Firefox (v122+) ou Microsoft Edge (v120+), dispensando qualquer tipo de instalação local. O desenvolvimento do *backend* em linguagens portáteis (Python ou Node.js) e a utilização do banco de dados PostgreSQL conferem independência de sistema operacional, simplificando uma eventual migração entre diferentes provedores de computação em nuvem. Complementarmente, a interface de usuário possui *design* responsivo compatível com resoluções de tela que variam de 1024px a 1920px (RNF-013), adequando-se perfeitamente aos diferentes monitores e terminais de trabalho utilizados na clínica.

6.7 Testabilidade

A divisão clara de responsabilidades arquiteturais simplifica a execução de testes unitários isolados para cada camada. A camada de negócios (serviços) foi projetada livre de acoplamentos diretos com elementos de infraestrutura, o que viabiliza a injeção de dependências e a criação de objetos simulados (*mocks*) durante a validação do código. Estabeleceu-se uma meta mínima de cobertura de testes de 70% para as regras de negócio (RNF-018). Adicionalmente, todos os *endpoints* da API estão documentados em conformidade com a especificação OpenAPI 3.0 (RNF-019), o que propicia a execução automatizada de testes de contrato e integração.

6.8 Resumo de Tradeoffs

Tabela 13 – Tradeoffs arquiteturais

Atributo favorecido	Atributo impactado	Natureza do tradeoff	Decisão tomada
Segurança	Desempenho	JWT, RBAC e criptografia adicionam verificação por requisição.	Segurança priorizada.
Modificabilidade	Desempenho	Camadas adicionam chamadas intermediárias.	Manutenibilidade priorizada.
Disponibilidade	Custo	Alta disponibilidade aumenta custo de infraestrutura.	Single-AZ no escopo acadêmico.
Portabilidade	Desempenho	REST/JSON possui overhead em relação a RPC binário.	Interoperabilidade priorizada.
Segurança	Escalabilidade	JWT stateless facilita escalabilidade horizontal e controle de acesso.	JWT com RBAC adotado.

7 PADRÕES ARQUITETURAIS ADOTADOS

A arquitetura do MedClinic foi definida a partir da combinação de dois padrões arquiteturais complementares: Cliente-Servidor e Camadas. Essa escolha é adequada ao contexto do projeto, pois favorece a organização do sistema, a separação de responsabilidades e a manutenção do código ao longo do desenvolvimento. Além disso, trata-se de uma abordagem compatível com sistemas de informação web de médio porte, especialmente quando há necessidade de controle de acesso, persistência segura de dados e evolução gradual das funcionalidades.

7.1 Cliente-servidor

O padrão Cliente-Servidor organiza o sistema em dois papéis principais. O cliente, representado pelo navegador web do usuário, realiza solicitações ao servidor. O servidor, por sua vez, processa essas solicitações, aplica as regras de negócio necessárias e retorna as respostas correspondentes.

No MedClinic, a comunicação entre cliente e servidor ocorre exclusivamente por meio de uma API RESTful sobre HTTPS, utilizando JSON como formato para troca de dados. Essa decisão contribui para uma integração mais simples entre as partes do sistema e permite que a interface web consuma os serviços disponibilizados pelo *backend* de forma padronizada.

Esse padrão foi adotado pelos seguintes motivos:

- a) O sistema precisa ser acessado a partir de diferentes estações de trabalho, sem necessidade de instalação local;
- b) A centralização da lógica de negócio no servidor garante maior consistência no processamento das regras do sistema;
- c) A separação entre cliente e servidor facilita o controle de acesso e reduz a duplicação de regras no lado do cliente;
- d) A independência entre *frontend* e *backend* permite que a interface evolua sem exigir alterações diretas na lógica de negócio.

Uma característica importante dessa arquitetura é que o servidor não mantém estado de sessão entre as requisições. O estado de autenticação é transportado pelo próprio cliente por meio de JWT (*JSON Web Token*). Dessa forma, o *backend* permanece *stateless*, o que facilita a escalabilidade horizontal do sistema.

7.2 Camadas

No lado do servidor, o MedClinic segue o padrão de três camadas lógicas, também conhecido como *Three-Tier*. Cada camada possui uma responsabilidade bem definida e se comunica com as demais por meio de interfaces controladas. Essa separação melhora a organização interna do sistema e reduz o acoplamento entre as partes da aplicação.

7.2.1 Camada de apresentação

A camada de apresentação corresponde ao *frontend* da aplicação. Ela é responsável pela interface gráfica do usuário, pelas validações iniciais de formulário, pelo consumo da API REST e pela renderização dos dados recebidos do servidor.

Essa camada é implementada com tecnologias web, como HTML5, CSS3 e JavaScript, com apoio de frameworks visuais como Bootstrap 5 ou Tailwind CSS. Não há regras de negócio nessa camada, pois sua função principal é intermediar a interação entre o usuário e os serviços disponibilizados pelo sistema.

7.2.2 Camada de negócio

A camada de negócio corresponde ao *backend* ou servidor de aplicação. Ela concentra as principais regras do sistema, incluindo agendamento de consultas, controle de acesso baseado em papéis, geração de prescrições, cálculos financeiros e registros de auditoria.

Essa camada é implementada em Python, utilizando Django ou Flask, e expõe uma API REST versionada por meio do caminho `/api/v1/`. Os módulos funcionais do sistema, como Acesso, Cadastros, Atendimento, Prontuário, Farmácia e Financeiro, são organizados como conjuntos coesos de serviços, o que facilita a manutenção e a evolução gradual da aplicação.

7.2.3 Camada de dados

A camada de dados é responsável pela persistência e recuperação das informações do sistema. O banco de dados adotado é o PostgreSQL v16.x, hospedado no AWS RDS. O acesso aos dados pela camada de negócio ocorre exclusivamente por meio de ORM (*Object-Relational Mapping*), evitando o uso de SQL diretamente nas regras de negócio.

Além do banco de dados relacional, arquivos como atestados e documentos gerados pelo sistema são armazenados no AWS S3. O acesso a esses arquivos é controlado pela camada de negócio por meio de URLs pré-assinadas, garantindo maior segurança no armazenamento e na recuperação dos documentos.

A comunicação entre as camadas segue uma hierarquia estrita. A camada de apresentação não acessa diretamente a camada de dados. Todo acesso deve passar obrigatoriamente pela camada de negócio, garantindo que as regras de validação, controle de acesso e auditoria sejam aplicadas antes de qualquer operação sobre os dados.

Justificativa para o uso de Three-Tier em vez de microserviços: Considerando o prazo de 14 semanas, a equipe formada por 6 desenvolvedores júnior e a complexidade envolvida na orquestração de microserviços, o padrão Three-Tier apresenta melhor equilíbrio entre modularidade, manutenibilidade e viabilidade de implementação. A organização modular do *backend* também permite que, futuramente, módulos específicos sejam migrados para microserviços sem exigir uma reestruturação completa da aplicação.

7.3 Estilo complementar: API REST

A comunicação entre a camada de apresentação e a camada de negócio segue o estilo arquitetural REST (*Representational State Transfer*). Nesse modelo, os recursos do sistema são identificados por URIs hierárquicas, como `/api/v1/pacientes`, `/api/v1/consultas` e `/api/v1/estoque`.

As operações sobre esses recursos são realizadas por meio dos verbos HTTP, como GET, POST, PUT e DELETE. Todas as respostas são retornadas em formato JSON e utilizam códigos de status HTTP semânticos, permitindo que o cliente compreenda corretamente o resultado de cada requisição.

A API também é versionada, conforme indicado pelo prefixo `/api/v1/`. Essa prática contribui para a retrocompatibilidade do sistema durante atualizações, pois novas versões podem ser criadas sem comprometer imediatamente os clientes que ainda utilizam versões anteriores da interface de comunicação.

8 MODELO DE ANÁLISE BCE (BOUNDARY-CONTROL-ENTITY)

O modelo BCE (*Boundary-Control-Entity*) é utilizado na análise orientada a objetos para organizar as classes do sistema de acordo com suas responsabilidades. No MedClinic, essa classificação permite separar os elementos de interface, controle de fluxo e domínio de dados, contribuindo para maior coesão, menor acoplamento e melhor manutenção da arquitetura.

A adoção do BCE também facilita a evolução do sistema, pois mudanças em uma parte da aplicação tendem a ficar restritas ao seu respectivo tipo de classe. Assim, alterações nas telas não afetam diretamente as regras de negócio, e mudanças nas regras de negócio não exigem, necessariamente, modificações nas entidades do domínio.

Tabela 14 – Estereótipos BCE

Estereótipo	Símbolo / Cor	Responsabilidade
Boundary	«boundary» - laranja	Representa a comunicação com os atores do sistema. Atua como ponto de entrada e saída de informações, sem conter lógica de negócio. No MedClinic, corresponde às telas, formulários e endpoints da API.
Control	«control» - roxo	Coordena a execução dos casos de uso e concentra a lógica de negócio. Atua como intermediária entre as classes Boundary e Entity, organizando as ações do usuário e tratando exceções.
Entity	«entity» - verde	Representa os conceitos centrais do domínio do negócio. Armazena informações persistentes e pode participar de diferentes casos de uso do sistema.

Cada objeto deve possuir uma responsabilidade principal bem definida, evitando o acúmulo de funções em uma mesma classe. Essa separação torna o projeto mais claro e reduz

impactos causados por alterações futuras. Dessa maneira, mudanças na interface, representada pelas classes *Boundary*, não afetam diretamente a lógica de negócio, concentrada nas classes *Control*. Da mesma forma, alterações nas regras de negócio tendem a não comprometer diretamente as classes de domínio, representadas pelas classes *Entity*.

O diagrama de classes completo, contendo os estereótipos BCE aplicados às classes do MedClinic, será entregue como artefato separado na fase de Design de Análise (T3).

8.1 Princípios de especificação BCE

A especificação BCE segue o diagrama de classes de nível de projeto elaborado pelo grupo, identificado como *Diagrama_De_Classe_Nível_De_Projeto v1*. Para manter a padronização dos artefatos, foi adotada uma nomenclatura direta para os três tipos de classe.

As classes *Boundary* utilizam o prefixo *Tela*, seguido do domínio funcional correspondente. As classes *Control* utilizam o prefixo *CTR*, seguido do domínio ou caso de uso relacionado. As classes *Entity* recebem o nome do próprio conceito de domínio que representam.

O mapeamento objeto-relacional elaborado pelo grupo é utilizado como referência principal para a definição dos atributos das entidades. Com isso, as classes *Entity* permanecem alinhadas às tabelas previstas para a persistência dos dados no sistema.

- a) **Classes *Boundary*:** representam as interfaces de interação com o usuário. Não executam regras de negócio e atuam apenas na captura, apresentação ou encaminhamento de informações;
- b) **Classes *Control*:** coordenam a lógica de execução de um domínio ou caso de uso. Recebem as solicitações das classes *Boundary*, acionam as classes *Entity* necessárias e tratam exceções do fluxo;
- c) **Classes *Entity*:** representam os conceitos persistentes do domínio do negócio. Seus atributos são definidos a partir do mapeamento objeto-relacional do grupo, conforme o documento de tabelas correspondente.

8.2 Módulo de Gestão de Acesso

Tabela 15 – BCE do módulo de Gestão de Acesso

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Login	Captura credenciais e repassa ao controle de autenticação.	UC-01
Boundary	Tela Redefinição Senha	Captura nova senha e token de recuperação.	UC-02
Control	CTR Autenticação	Valida credenciais, gera JWT, gerencia refresh token e aplica RBAC.	Todos os UCs de login

Tipo	Classe	Responsabilidade principal / atributos	UCs
Control	CTR Redefinição Senha	Gera link temporário, valida expiração e atualiza senha com hash.	UC-02
Entity	Recepcionista	idRecepcionista, nome, cpf, email, senha, telefone.	UC-05, UC-06, UC-07, UC-21
Entity	Medico	idMedico, nome, cpf, email, senha, telefone, crm, especializacao, idAgenda.	UC-11, UC-12, UC-18, UC-19
Entity	Enfermeiro	idEnfermeiro, nome, cpf, email, senha, telefone, coren.	UC-11, UC-17
Entity	Farmaceutico	idFarmaceutico, nome, cpf, telefone, email, senha, crf.	UC-11, UC-13, UC-14, UC-16
Entity	Gestor	idGestor, nome, cpf, telefone, email, senha.	UC-08, UC-09, UC-10, UC-20

8.3 Módulo de Gestão de Cadastros

Tabela 16 – BCE do módulo de Gestão de Cadastros

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Paciente	Cadastro, edição, visualização e busca de pacientes.	UC-05, UC-06, UC-07
Control	CTR Paciente	Valida CPF, persiste cadastro, executa busca por CPF/nome e registra alterações.	UC-05, UC-06, UC-07
Entity	Paciente	idPaciente, nome, cpf, telefone, dataNascimento, email, senha, idRecepcionista, idAgenda, idProntuario.	UC-01, UC-02, UC-06, UC-07

8.4 Módulo de Fluxo de Atendimento

Tabela 17 – BCE do módulo de Fluxo de Atendimento

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Consulta	Agenda, reagenda e cancela consultas; exibe disponibilidade.	UC-04, UC-05
Boundary	Tela Medico	Exibe agenda do dia, status da consulta e acesso ao prontuário.	UC-11, UC-12, UC-18, UC-19
Control	CTR Consulta	Verifica disponibilidade na Agenda, persiste Consulta em transação, cancela e reagenda.	UC-19, UC-21
Control	CTR Medico	Consulta agenda por data, altera status e aciona prontuário.	UC-11, UC-12, UC-18, UC-19

Tipo	Classe	Responsabilidade principal / atributos	UCs
Entity	Agenda	idAgenda, data, horariosProfissionais, ocupacao-Salas.	UC-04, UC-05, UC-19
Entity	Consulta	idConsulta, data, horario, status, idSala, idAgenda, idPaciente, idMedico, idProntuario.	UC-05, UC-19

8.5 Módulo de Prontuário Eletrônico

Tabela 18 – BCE do módulo de Prontuário Eletrônico

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Prontuario	Exibe prontuário completo ao médico, enfermeiro ou farmacêutico autorizado. Permite registrar evolução, emitir prescrição e atestado.	UC-11
Control	CTR Prontuario	Verifica acesso por perfil (tabelas Medico_Prontuario, Enfermeiro_Prontuario, Farmaceutico_Prontuario), inicializa prontuário, atualiza evolução (diagnostico, observacoes, prescricaoAtiva), obtém histórico clínico.	UC-11, UC-12
Entity	Prontuario	idProntuario, dataCriacao, diagnostico, observacoes, prescricaoAtiva. Tabelas associativas: Medico_Prontuario, Enfermeiro_Prontuario, Farmaceutico_Prontuario. (ORM: tabela Prontuario).	UC-06, UC-11, UC-12

8.6 Módulo de Estoque de Materiais

Tabela 19 – BCE do módulo de Estoque de Materiais

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Material	Interface do enfermeiro para visualizar estoque de materiais e registrar consumo. Interface do gestor para inserir novos lotes e ajustar limites.	UC-15, UC-17, UC-20
Control	CTR Materiais	Consulta materiais disponíveis, registra consumo de material com idEnfermeiro para rastreabilidade, verifica nível mínimo e atualiza quantidades.	UC-15, UC-17, UC-20
Entity	Material	idMaterial, nome, descricao, numeroLote, quantidadeEstoque, quantidadeMinima, dataValidade. ORM: tabela Material.	UC-17, UC-20

Tipo	Classe	Responsabilidade principal / atributos	UCs
Entity	Material_Consumido	idConsumo, quantidade, idMaterial, idEnfermeiro. Registra o consumo de material realizado pelo enfermeiro. ORM: tabela Material_Consumido.	UC-17
Entity	Gestor_Material	idGestor, idMaterial. Tabela associativa de supervisão do gestor sobre materiais. ORM: tabela Gestor_Material.	UC-20

8.7 Módulo de Estoque de Medicamentos e Dispensação

Tabela 20 – BCE do módulo de Estoque de Medicamentos e Dispensação

Tipo	Classe	Responsabilidade principal / atributos	UCs
Boundary	Tela Medicamento	Interface do farmacêutico para visualizar estoque de medicamentos, consultar prescrições ativas no Prontuário e registrar dispensação.	UC-14, UC-16, UC-20
Control	CTR Medicamento	Consulta medicamentos disponíveis, verifica disponibilidade antes da dispensação, registra Medicamento_Dispensado com idFarmaceutico e idProntuario, verifica nível mínimo e notifica gestor para reposição.	UC-13, UC-16, UC-20
Entity	Medicamento	idMedicamento, nome, descricao, numeroLote, quantidadeEstoque, quantidadeMinima, dataValidade. ORM: tabela Medicamento.	UC-14, UC-16, UC-20
Entity	Medicamento_Dispensado	idDispensacao, quantidade, idFarmaceutico, idMedicamento, idProntuario. Rastreabilidade da dispensação vinculada ao prontuário. ORM: tabela Medicamento_Dispensado.	UC-16
Entity	Gestor_Medicamento	idGestor, idMedicamento. Tabela associativa de supervisão do gestor sobre medicamentos. ORM: tabela Gestor_Medicamento.	UC-20

8.8 Correspondência BCE e Three-Tier

Tabela 21 – Correspondência BCE e camadas

BCE	Camada Three-Tier	Artefato de código	Exemplo
Tela / Boundary	Apresentação (Frontend)	Páginas HTML/JS por perfil; controllers.py (endpoints REST).	Tela Consulta → /frontend/src/pages/consulta/ + /backend/app/modules/atendimento/controllers.py
CTR / Control	Negócio (Backend / Services)	services.py de cada módulo.	CTR EstoqueMed → /backend/app/modules/farmacia/services.py
Entity	Dados (Backend / Repository + DB)	models.py (ORM) + repositories.py + tabelas PostgreSQL.	Medicamento_Dispensado → tabela.

Observação. A Seção 9.2 apresenta como os módulos lógicos da arquitetura mapeiam para as tabelas ORM definidas pelo grupo no documento de Mapeamento Objeto-Relacional.

9 VISÃO LÓGICA (LOGICAL VIEW)

9.1 Propósito e preocupações arquiteturais

A Visão Lógica apresenta a organização funcional do MedClinic, com foco na modificabilidade, separação de responsabilidades, modularidade e controle da complexidade. Essa visão descreve como o sistema é estruturado em pacotes, módulos e classes principais, evidenciando a distribuição das responsabilidades entre os componentes do *backend*.

No contexto do MedClinic, essa organização favorece a manutenção do sistema, pois cada módulo concentra responsabilidades específicas e se comunica com os demais de forma controlada. Dessa maneira, busca-se reduzir o acoplamento entre as partes da aplicação e aumentar a coesão interna dos módulos.

Esta visão foi definida a partir das entidades presentes no mapeamento objeto-relacional do projeto. Portanto, não são introduzidas entidades que não estejam representadas no modelo relacional, preservando a consistência entre a arquitetura lógica e a camada de persistência.

9.2 Pacotes e módulos principais

O sistema MedClinic é organizado em seis módulos funcionais no *backend*, definidos a partir do mapeamento objeto-relacional elaborado pelo grupo. As entidades listadas correspondem exclusivamente às tabelas ORM utilizadas para representar os conceitos do domínio.

Algumas entidades aparecem em mais de um módulo porque possuem papéis diferentes na arquitetura. Por exemplo, as tabelas de perfis de usuário são utilizadas pelo módulo de

Acesso para autenticação e autorização, mas também representam dados cadastrais dos usuários do sistema.

Tabela 22 – Módulos lógicos do MedClinic

Módulo	Entidades ORM Principais	Responsabilidades
Acesso	Recepcionista, Paciente, Medico, Enfermeiro, Farmaceutico, Gestor	Autenticar usuários a partir das credenciais armazenadas em suas respectivas tabelas; aplicar controle de acesso por perfil; permitir redefinição de senha e separação das permissões entre recepcionista, paciente, médico, enfermeiro, farmacêutico e gestor.
Cadastros	Paciente, Recepcionista, Medico, Enfermeiro, Farmaceutico, Gestor	Manter os dados cadastrais dos usuários e profissionais da clínica. A entidade Paciente possui vínculos com Recepcionista, Agenda e Prontuario por meio dos atributos idRecepcionista, idAgenda e idProntuario.
Agenda e Atendimento	Agenda, Consulta, Recepcionista_Agenda	Controlar a agenda da clínica e o registro das consultas. A entidade Consulta referencia Agenda, Paciente, Medico e Prontuario. A tabela Recepcionista_Agenda representa a associação entre recepcionistas e agendas.
Prontuário Eletrônico	Prontuario, Medico_Prontuario, Enfermeiro_Prontuario, Farmaceutico_Prontuario	Registrar informações clínicas do paciente, incluindo dataCriacao, diagnostico, observacoes, historicoEvolucoes e prescricaoAtiva. O acesso ao prontuário por médicos, enfermeiros e farmacêuticos é representado pelas tabelas associativas correspondentes.
Estoque de Materiais	Material, Material_Consumido, Gestor_Material	Controlar materiais utilizados na clínica, incluindo lote, quantidade em estoque, quantidade mínima e validade. A tabela Material_Consumido registra o consumo de materiais por enfermeiro, enquanto Gestor_Material representa a associação administrativa entre gestor e material.

Módulo	Entidades ORM Principais	Responsabilidades
Estoque de Medicamentos e Dispensação	Medicamento, Medicamento_Dispensado, Gestor_Medicamento	Controlar medicamentos, seus lotes, quantidades, validade e dispensações. A tabela Medicamento_Dispensado registra a dispensação de medicamentos por farmacêutico e vincula essa dispensação ao prontuário do paciente. A tabela Gestor_Medicamento representa a associação administrativa entre gestor e medicamento.

9.3 Estrutura em camadas dentro do backend

Dentro de cada módulo, o código é organizado em três subcamadas horizontais. Essa divisão promove a separação de responsabilidades, facilita a realização de testes unitários e reduz o impacto de mudanças em partes específicas do sistema.

- a) **Controllers (Rotas / Tela):** recebem as requisições HTTP, validam os dados de entrada, delegam o processamento aos serviços e formatam as respostas retornadas ao cliente. Correspondem ao estereótipo *Tela (Boundary)* no modelo BCE do grupo;
- b) **Services (CTR / Serviços de Negócio):** implementam a lógica de negócio de cada domínio. Correspondem ao estereótipo *CTR (Control)* no modelo BCE do grupo. Devem permanecer independentes do protocolo HTTP e da tecnologia de persistência;
- c) **Repositories (Acesso a Dados):** encapsulam o acesso ao banco de dados por meio do ORM. Essa é a única subcamada que conhece diretamente o esquema relacional definido no mapeamento objeto-relacional do grupo.

Essa organização garante que alterações no esquema ORM fiquem concentradas nos repositórios, que mudanças nas regras de negócio sejam tratadas nos *services* e que ajustes na interface web sejam isolados nos *controllers*. Com isso, o sistema mantém uma estrutura mais previsível e adequada à evolução dos requisitos.

9.4 Principais relacionamentos entre módulos

As dependências entre os módulos derivam principalmente das chaves estrangeiras e tabelas associativas definidas no mapeamento objeto-relacional. Esses relacionamentos indicam como os módulos compartilham informações e como as operações de um domínio podem depender de entidades pertencentes a outro domínio.

- a) **Cadastros e Acesso:** as entidades Recepcionista, Paciente, Medico, Enfermeiro, Farmaceutico e Gestor armazenam dados cadastrais e credenciais de acesso, como email e senha;

- b) Cadastros e Atendimento:** a entidade *Paciente* possui vínculo com *Recepcionista* e *Agenda* por meio dos atributos *idRecepcionista* e *idAgenda*. A entidade *Medico* também se vincula à agenda por meio de *idAgenda*;
- c) Atendimento e Agenda:** a entidade *Consulta* referencia a tabela *Agenda* por meio do atributo *idAgenda*. Além disso, a tabela *Recepcionista_Agenda* associa recepcionistas às agendas;
- d) Atendimento e Cadastros:** a entidade *Consulta* referencia *Paciente* e *Medico* por meio dos atributos *idPaciente* e *idMedico*;
- e) Atendimento e Prontuário:** a entidade *Consulta* referencia *Prontuario* por meio do atributo *idProntuario*. Além disso, a entidade *Paciente* também possui vínculo direto com *Prontuario* por meio de *idProntuario*;
- f) Prontuário e Perfis Profissionais:** as tabelas *Medico_Prontuario*, *Enfermeiro_Prontuario* e *Farmaceutico_Prontuario* representam as associações entre prontuários e os profissionais autorizados a consultá-los ou atualizá-los;
- g) Materiais e Enfermagem:** a entidade *Material_Consumido* referencia *Material* e *Enfermeiro* por meio dos atributos *idMaterial* e *idEnfermeiro*, permitindo rastrear o consumo de materiais;
- h) Medicamentos, Farmácia e Prontuário:** a entidade *Medicamento_Dispensado* referencia *Farmaceutico*, *Medicamento* e *Prontuario* por meio dos atributos *idFarmaceutico*, *idMedicamento* e *idProntuario*, permitindo rastrear a dispensação de medicamentos;
- i) Gestor e Estoques:** as tabelas *Gestor_Material* e *Gestor_Medicamento* associam o gestor aos materiais e medicamentos sob controle administrativo.

A entidade *Prontuario* não possui, no mapeamento objeto-relacional atual, chaves estrangeiras diretas para *Paciente*, *Medico*, *Enfermeiro*, *Farmaceutico* ou *Consulta*. Esses vínculos são representados por outras entidades: *Paciente* e *Consulta* referenciam *Prontuario*, e as tabelas associativas ligam o prontuário aos profissionais autorizados.

Também não há, no mapeamento atual, uma entidade separada para sala. O atributo *idSala* permanece como atributo da entidade *Consulta*.

9.5 Architecture Models da Visão Lógica (ISO 42010, Seção 5.6)

Tabela 23 – Modelos da visão lógica

ID	Model Kind	Conteúdo	Versão
ML-01	MK-L1 — Diagrama de Pacotes e Módulos	Tabela de 6 módulos funcionais com entidades ORM e responsabilidades, conforme a Seção 9.2.	v1.1 — Maio/2026
ML-02	MK-L2 — Tabela BCE por Módulo	Classes Boundary, Control e Entity organizadas por módulo funcional, mantendo correspondência com as entidades existentes no mapeamento objeto-relacional.	v1.1 — Maio/2026
ML-03	MK-L3 — Relacionamentos entre Módulos	Descrição textual das dependências entre os 6 módulos lógicos a partir das chaves estrangeiras e tabelas associativas do ORM, conforme a Seção 9.4.	v1.1 — Maio/2026

9.6 Inconsistências conhecidas (ISO 42010, Seção 5.7.1)

Conforme exigido pela Cláusula 5.7.1 da ISO/IEC/IEEE 42010:2011, registra-se que nenhuma inconsistência conhecida foi identificada nesta visão arquitetural ou entre esta visão e as demais visões do projeto.

As dependências entre os módulos lógicos, descritas em ML-03, permanecem consistentes com as classes *Entity* definidas no modelo BCE, apresentado em ML-02. Além disso, essas dependências também estão alinhadas aos repositórios descritos no *Viewpoint* de Implementação, conforme indicado na Seção 11.

10 VISÃO DE PROCESSOS (PROCESS VIEW)

10.1 Propósito e preocupações arquiteturais

A Visão de Processos descreve o comportamento do MedClinic em tempo de execução, considerando aspectos como desempenho, concorrência, integridade transacional, tolerância a falhas e disponibilidade. Essa visão é especialmente relevante para os perfis de Médico, Farmacêutico, Recepcionista e Administrador de TI, pois esses usuários dependem de respostas consistentes, seguras e disponíveis durante a execução das operações do sistema.

Essa visão complementa a Visão Lógica ao demonstrar como os módulos e componentes definidos estruturalmente operam durante o uso da aplicação. Dessa forma, são considerados os processos principais, a comunicação entre componentes, o tratamento de requisições concorrentes e os mecanismos adotados para preservar a integridade dos dados.

10.2 Processos e *threads* principais

O MedClinic opera com diferentes processos em tempo de execução, cada um responsável por uma parte específica do funcionamento da aplicação. A separação desses processos contribui para maior organização operacional, melhor controle de concorrência e maior resiliên-

cia diante de falhas.

- a) **Processo do Web Server (Gunicorn/uWSGI):** gerencia o conjunto de *workers* responsáveis por atender requisições HTTP concorrentes. Cada *worker* processa uma requisição de forma independente, sem compartilhamento de estado em memória. Essa característica é viabilizada pelo uso de JWT, mantendo o *backend stateless*. A quantidade de *workers* pode ser ajustada conforme a carga da instância EC2;
- b) **Processo de Banco de Dados (PostgreSQL RDS):** gerencia conexões concorrentes por meio de um *pool* de conexões, como PgBouncer ou o próprio mecanismo do ORM. As transações ACID são utilizadas para garantir a integridade referencial entre entidades relacionadas. Por exemplo, a dispensação de um medicamento deve registrar a movimentação e decrementar o estoque dentro da mesma transação;
- c) **Processo de Armazenamento de Arquivos (AWS S3):** gerencia o armazenamento de documentos vinculados ao prontuário. O envio de arquivos ocorre de forma não bloqueante. O *backend* gera uma URL pré-assinada e a retorna ao cliente, permitindo que o navegador envie o arquivo diretamente ao S3, sem transferir o conteúdo pelo servidor da aplicação;
- d) **Processo de Envio de E-mails (AWS SES / SMTP):** executa o envio assíncrono de mensagens relacionadas a eventos do sistema, como confirmação de agendamento, comprovante de pagamento e emissão de atestado médico. O *backend* registra a solicitação de envio sem bloquear a resposta principal ao cliente.

10.3 Comunicação entre componentes

O estilo de comunicação predominante no MedClinic é Cliente-Servidor síncrono, realizado por meio de REST sobre HTTP. Esse modelo é adequado para operações interativas que exigem resposta imediata ao usuário, como consultas, cadastros, autenticação e agendamento.

Para operações que não exigem retorno imediato, como envio de e-mails e armazenamento de arquivos, é adotado um modelo assíncrono. Essa estratégia evita o bloqueio da *thread* principal de atendimento da requisição e reduz o tempo de resposta percebido pelo usuário.

O ciclo de vida de uma requisição típica ocorre da seguinte forma:

- a) O cliente envia uma requisição HTTPS contendo o token JWT no cabeçalho *Authorization*;
- b) O *middleware* de autenticação valida o JWT e identifica o perfil do usuário;
- c) O *middleware* de autorização RBAC verifica se o perfil possui permissão para acessar o recurso solicitado;
- d) O *controller* recebe a requisição validada e delega o processamento ao serviço correspondente;

- e) O serviço executa a regra de negócio e interage com o repositório quando há necessidade de persistência ou consulta de dados;
- f) O repositório executa a operação no banco de dados por meio do ORM, preferencialmente dentro de uma transação;
- g) O sistema retorna uma resposta em JSON ao cliente, acompanhada de um código de status HTTP semântico.

10.4 Concorrência, integridade e falhas

As operações que envolvem múltiplas entidades são executadas dentro de transações de banco de dados. Essa abordagem garante que todas as etapas de uma operação sejam concluídas com sucesso antes da confirmação definitiva dos dados. Caso ocorra falha em qualquer etapa, a transação é revertida automaticamente, preservando a consistência do sistema.

Um exemplo desse comportamento ocorre na dispensação de medicamentos. O sistema deve registrar a dispensação, associá-la ao prontuário do paciente e atualizar a quantidade disponível em estoque. Essas ações precisam ocorrer de forma conjunta, pois a conclusão parcial da operação poderia gerar inconsistência entre o estoque e o histórico clínico.

A tolerância a falhas em serviços externos é tratada por meio de tentativas controladas de repetição. Operações como envio de e-mail e comunicação com o S3 podem ser repetidas até três vezes, utilizando *backoff* exponencial. Caso a falha persista, o erro é registrado em log e a operação principal pode ser concluída com uma resposta de sucesso parcial, indicando que a ação principal foi realizada, mas que a notificação ou etapa auxiliar permaneceu pendente.

Essa estratégia evita que falhas em serviços externos, como AWS SES ou AWS S3, bloqueiem operações clínicas críticas do MedClinic. Com isso, o sistema preserva a continuidade das atividades essenciais, ao mesmo tempo em que mantém rastreabilidade para posterior tratamento das falhas registradas.

10.5 Architecture Models desta Visão (ISO 42010, Seção 5.6)

Tabela 24 – Modelos da visão de processos

ID	Model Kind	Conteúdo	Versão
MP-01	MK-P1 — Descrição de Processos e Threads	Lista dos 4 processos em execução com modo de comunicação e política de falha (Seção 10.2)	v1.0 — Maio/2026
MP-02	MK-P2 — Fluxo de Requisição HTTP	Ciclo de vida de uma requisição em 7 passos: cliente → middleware JWT → RBAC → controller → service → repository → resposta (Seção 10.3)	v1.0 — Maio/2026

10.6 Inconsistências conhecidas

Não foram identificadas inconsistências conhecidas nesta visão arquitetural. Os processos descritos no modelo MP-01 correspondem aos componentes implantados nos nós definidos no *Viewpoint* Físico, conforme apresentado na Seção 12. Além disso, o fluxo de requisição descrito em MP-02 está alinhado aos módulos definidos no *Viewpoint* Lógico, apresentado na Seção 9, e aos artefatos de código descritos no *Viewpoint* de Implementação, apresentado na Seção 11.

11 VISÃO DE IMPLEMENTAÇÃO (DEVELOPMENT VIEW)

11.1 Propósito e preocupações arquiteturais

A Visão de Implementação descreve a organização interna dos artefatos de software do MedClinic, incluindo diretórios, módulos, arquivos, bibliotecas e componentes de código. Essa visão está relacionada à modificabilidade, testabilidade, organização da equipe, reuso e facilidade de implantação.

No contexto do projeto, essa visão serve como base para a divisão das tarefas de desenvolvimento, para o controle da evolução do código-fonte e para a definição dos artefatos que serão implantados no ambiente de execução. Além disso, permite relacionar a estrutura física do repositório aos módulos definidos na Visão Lógica.

11.2 Organização do repositório

O código-fonte do MedClinic é versionado em um repositório Git hospedado no GitHub. A organização do repositório segue uma divisão por responsabilidade, separando a aplicação servidor, a interface web, os artefatos de infraestrutura e a documentação do projeto.

Tabela 25 – Estrutura de implementação

Diretório / Artefato	Conteúdo e Responsabilidade
/backend	Módulo 1 — Aplicação servidor em Python. Contém toda a lógica de negócio, API REST e acesso a dados.
/backend/app/modules/<módulo>/	Subdiretório por módulo funcional (acesso, cadastros, atendimento, prontuario, farmacia, financeiro). Cada um contém controllers.py, services.py, repositories.py e models.py.
/backend/app/middleware/	Middleware transversal: autenticação JWT, autorização RBAC, rate limiting, logging de auditoria.
/backend/app/config/	Configurações de ambiente, conexão com banco de dados, integração com AWS (KMS, SES, S3). Lê variáveis do AWS Secrets Manager em produção.
/backend/tests/	Testes unitários e de integração, organizados espelhando a estrutura de módulos. Meta de cobertura: 70%.

Diretório / Artefato	Conteúdo e Responsabilidade
/frontend	Módulo 2 — Interface web em JavaScript/HTML5/CSS3. Contém todas as telas por perfil de usuário.
/frontend/src/pages/<perfil>/	Telas agrupadas por perfil (paciente, recepcao, medico, enfermeiro, farmacia, gestor).
/frontend/src/components/	Componentes reutilizáveis do design system: botões, modais, tabelas, formulários, spinners.
/frontend/src/services/	Camada de abstração para chamadas à API REST do backend (fetch/axios). Centraliza URLs e tratamento de erros HTTP.
/infra	Módulo 3 — Scripts de infraestrutura: migrations do banco de dados, configurações de CI/CD, scripts de deploy e backup.
/infra/migrations/	Arquivos de migração do banco de dados versionados cronologicamente. Executados pelo ORM na inicialização.
/docs	Artefatos de documentação: SRS, casos de uso, documento de arquitetura, protótipos Figma exportados.

11.3 Subcamadas do backend

A implementação do *backend* é organizada em subcamadas, seguindo a separação de responsabilidades definida no modelo BCE. Essa estrutura facilita a manutenção do código, a realização de testes e o isolamento de mudanças.

Tabela 26 – Subcamadas de implementação

Subcamada	Correspondência BCE	Responsabilidade
<i>Controllers</i>	<i>Boundary</i>	Recebem requisições, validam entradas e formatam respostas.
<i>Services</i>	<i>Control</i>	Executam regras de negócio e coordenam os casos de uso.
<i>Repositories</i>	Acesso a dados das <i>Entities</i>	Encapsulam consultas ao ORM e operações transacionais.
<i>Models</i>	<i>Entity</i>	Representam as tabelas definidas no mapeamento objeto-relacional.

Os *controllers* concentram a comunicação com a camada de apresentação, enquanto os *services* tratam as regras de negócio de cada domínio. Os *repositories* isolam o acesso ao banco de dados, e os *models* representam os conceitos persistentes do domínio. Com isso, alterações na interface, nas regras de negócio ou no esquema de dados podem ser tratadas em pontos específicos da aplicação.

11.4 Dependências tecnológicas

As dependências tecnológicas são organizadas por camada, com versões controladas para favorecer a reprodutibilidade do ambiente de desenvolvimento e implantação.

- a) **Backend:** utiliza Python 3.12.x, com dependências gerenciadas por pip e registradas em `requirements.txt`. As principais bibliotecas incluem framework web, ORM, driver PostgreSQL, SDK AWS, PyJWT e biblioteca para *hashing* de senhas;
- b) **Frontend:** utiliza HTML5, CSS3 e JavaScript, com apoio de Bootstrap 5 ou Tailwind CSS. As dependências são gerenciadas por npm e registradas em `package.json`;
- c) **Infraestrutura:** utiliza PostgreSQL 16.x no AWS RDS, Amazon EC2, Amazon S3, AWS Secrets Manager, AWS KMS, AWS CLI e GitHub Actions para integração contínua e implantação.

11.5 Pipeline de integração contínua

Cada *pull request* enviado para a branch principal aciona automaticamente o pipeline de integração contínua configurado no GitHub Actions. Esse processo tem como objetivo verificar a qualidade do código antes da incorporação das alterações ao repositório principal.

O pipeline executa as seguintes etapas:

- a) Análise estática do código do *frontend* com ESLint;
- b) Análise estática e formatação do código do *backend* com Flake8 e Black;
- c) Execução dos testes unitários, com cobertura mínima definida em 70%;
- d) Verificação de segurança das dependências com ferramentas como pip-audit e npm audit;
- e) Execução do *build* da aplicação.

Caso qualquer etapa apresente falha, o *pull request* é bloqueado para *merge*. Essa estratégia contribui para a integridade do repositório principal e reduz a chance de introdução de erros nas versões entregues.

11.6 Architecture Models desta Visão (ISO 42010, Seção 5.6)

Tabela 27 – Modelos da visão de implementação

ID	Model Kind	Conteúdo	Versão
MD-01	MK-D1 — Estrutura de Repositório	Tabela hierárquica de diretórios /backend, /frontend e /infra com responsabilidades (Seção 11.2)	v1.0 — Maio/2026
MD-02	MK-D2 — Dependências Tecnológicas	Stacks por camada: Python/pip (backend), Node.js/npm (frontend), AWS CLI/GitHub Actions (infra) com versões fixadas (Seção 11.3)	v1.0 — Maio/2026
MD-03	MK-D3 — Pipeline CI/CD	Sequência de 4 etapas do pipeline GitHub Actions: lint → testes → security audit → build (Seção 11.4)	v1.0 — Maio/2026

11.7 Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)

Não foram identificadas inconsistências conhecidas nesta visão arquitetural. A estrutura de diretórios descrita em MD-01 está alinhada aos módulos funcionais apresentados na Visão Lógica, conforme a Seção 9. Cada diretório localizado em /backend/app/modules/<modulo>/ corresponde a um módulo lógico do sistema. Além disso, os artefatos produzidos pelo pipeline descrito em MD-03 são implantados nos nós definidos na Visão Física, conforme a Seção 12.

12 VISÃO FÍSICA (DEPLOYMENT VIEW)

12.1 Propósito e preocupações arquiteturais

A Visão Física descreve como os artefatos de software do MedClinic são distribuídos na infraestrutura de execução. Essa visão trata de aspectos relacionados à disponibilidade, segurança de rede, custo de infraestrutura e facilidade de implantação.

No contexto do projeto, essa visão apresenta os nós físicos e serviços utilizados, os fluxos de comunicação entre eles e os ambientes previstos para desenvolvimento, homologação e produção. Dessa forma, é possível compreender como os componentes definidos nas visões anteriores são alocados em recursos de infraestrutura.

12.2 Nós de infraestrutura

A infraestrutura do MedClinic é baseada em serviços da AWS e em clientes web acessados por navegadores. Cada nó possui uma responsabilidade específica dentro do ambiente de execução, conforme apresentado na Tabela 28.

Tabela 28 – Nós físicos

Nó	Serviço AWS	Componentes Hospedados	Configuração
Servidor de Aplicação	EC2 t3.micro (min.)	Backend Python (Gunicorn/uWSGI), API REST /api/v1/	Ubuntu 24.04 LTS; acesso restrito por Security Group (porta 443 apenas).
Banco de Dados	RDS PostgreSQL 16.x (db.t3.micro)	Esquema relacional completo do MedClinic; backups automáticos diários	Single-AZ; backups retidos 7 dias; criptografia em repouso com AWS KMS.
Armazenamento de Arquivos	AWS S3 (bucket privado)	Atestados médicos, anexos de prontuário	Bucket privado; acesso via URLs pré-assinadas (validade 15 min); versionamento habilitado.
Gestão de Segredos	AWS Secrets Manager	Credenciais do banco, chaves de API, segredos de configuração	Rotação automática habilitada; acesso restrito à role IAM do EC2.
Gestão de Chaves	AWS KMS	Chaves AES-256-GCM para criptografia de dados sensíveis em repouso	Rotação anual automática; auditoria via CloudTrail.
E-mail Transacional	AWS SES	Envio de confirmações, comprovantes, atestados e links de redefinição de senha	Domínio verificado; monitoramento de bounces/complaints.
Monitoramento	AWS CloudWatch	Métricas de CPU, memória, latência de API, erros 5xx, disponibilidade	Alertas configurados para indisponibilidade > 5 min e CPU > 80%.
Clientes (Estações da Clínica)	Navegador web homologado	Frontend (HTML/CSS/JS estático), interface do usuário	Chrome 120+, Firefox 122+ ou Edge 120+; conexão de 10 Mbps mínimo.

12.3 Fluxos de comunicação entre nós

A comunicação entre os nós segue o princípio do menor privilégio. Assim, cada componente se comunica apenas com os serviços necessários para cumprir sua função, utilizando *Security Groups* e políticas IAM para controlar permissões e acessos.

- a) **Navegador do cliente para EC2:** a comunicação ocorre por HTTPS na porta 443. Esse é o único ponto de entrada da aplicação;
- b) **EC2 para RDS PostgreSQL:** a comunicação ocorre pela porta 5432, dentro da mesma VPC privada, sem exposição direta do banco de dados à internet;

- c) **EC2 para S3:** a comunicação ocorre por HTTPS, utilizando endpoints de serviço da AWS para acesso aos arquivos armazenados;
- d) **EC2 para SES:** a comunicação ocorre por HTTPS, permitindo o envio de e-mails transacionais do sistema;
- e) **EC2 para KMS e Secrets Manager:** a comunicação ocorre por HTTPS, permitindo operações de criptografia e recuperação segura de credenciais.

Não há comunicação direta do navegador com o RDS, com o Secrets Manager, com o KMS ou com os demais serviços internos da infraestrutura. O acesso aos recursos sensíveis é sempre intermediado pelo *backend*, garantindo a aplicação das regras de autenticação, autorização e auditoria.

A única exceção controlada ocorre no envio de arquivos para o Amazon S3. Nesse caso, o *backend* gera uma URL pré-assinada com validade de 15 minutos e a entrega ao cliente. Em seguida, o navegador realiza o upload diretamente para o S3, sem transferir o arquivo pelo servidor de aplicação. Essa estratégia reduz a carga sobre a instância EC2 e melhora o desempenho no envio de arquivos maiores.

12.4 Ambientes

O MedClinic prevê três ambientes principais, com estrutura semelhante para reduzir diferenças entre desenvolvimento, testes e implantação. Essa paridade facilita a identificação de falhas antes da disponibilização da aplicação em produção.

- a) **Desenvolvimento:** ambiente local utilizado pelos desenvolvedores, com PostgreSQL local e variáveis de ambiente definidas em arquivo `.env`. É destinado à implementação das funcionalidades e à execução de testes unitários;
- b) **Homologação:** ambiente separado, composto por instância EC2 e banco RDS próprios, utilizando dados de teste. É utilizado para testes de integração, validação de requisitos e revisões técnicas antes da implantação em produção;
- c) **Produção:** ambiente destinado ao uso real do sistema. Possui acesso restrito e implantação controlada por pipeline, após aprovação das alterações no ambiente de homologação.

12.5 Architecture Models desta Visão (ISO 42010, Seção 5.6)

Tabela 29 – Modelos da visão física

ID	Model Kind	Conteúdo	Versão
MF-01	MK-F1 — Tabela de Nós de Infraestrutura	9 nós AWS com serviços, componentes hospedados e configurações de segurança (Seção 12.2)	v1.0 — Maio/2026
MF-02	MK-F2 — Fluxo de Comunicação entre Nós	6 canais de rede com protocolos e portas (Seção 12.3); VPC privada para RDS; HTTPS para clientes	v1.0 — Maio/2026
MF-03	MK-F3 — Tabela de Ambientes	3 ambientes (Desenvolvimento, Homologação, Produção) com paridade estrutural e diferenças de escala (Seção 12.4)	v1.0 — Maio/2026

12.6 Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)

Foi identificada uma inconsistência potencial nesta visão arquitetural. O modelo MF-01 descreve o banco de dados Amazon RDS em configuração *Single-AZ*, enquanto a meta de disponibilidade mensal de 95%, apresentada na Seção 5, exige atenção operacional adicional.

Essa decisão é considerada aceitável no escopo acadêmico do projeto, desde que acompanhada por medidas complementares, como backups diários, monitoramento por Amazon CloudWatch e alertas de indisponibilidade. A inconsistência entre a meta de disponibilidade e a configuração inicial da infraestrutura está documentada e justificada na decisão arquitetural DA-006, conforme indicado na Seção 14.2.

Como resolução recomendada, propõe-se a migração do banco de dados para uma configuração *Multi-AZ* antes de uma eventual implantação em ambiente de produção real.

13 VISÃO DE CENÁRIOS (SCENARIO / USE CASE VIEW)

13.1 Propósito e preocupações arquiteturais

A Visão de Cenários demonstra como as visões Lógica, de Processos, de Implementação e Física atuam em conjunto durante a execução de funcionalidades relevantes do MedClinic. Essa visão permite verificar se as decisões arquiteturais adotadas são coerentes quando aplicadas a situações reais de uso do sistema.

Foram selecionados três cenários arquiteturalmente significativos, pois eles exercitam componentes importantes da arquitetura e envolvem preocupações críticas, como segurança, controle de acesso, integridade transacional, rastreabilidade, desempenho e disponibilidade.

13.2 Cenário 1: médico registra consulta e gera prescrição

Este cenário envolve os módulos de Atendimento, Prontuário e Farmácia. Ele exercita a autenticação do médico, o controle de acesso ao prontuário, o registro das informações clínicas e a geração de prescrição. A baixa de estoque não ocorre neste momento, sendo tratada posteriormente no cenário de dispensação de medicamento.

Tabela 30 – Cenário arquitetural: registro de consulta e prescrição

#	Passo do Cenário
1	Médico autenticado (JWT válido) acessa a agenda no frontend e seleciona a consulta em andamento.
2	Frontend envia GET /api/v1/consultas/{id}/prontuario com token JWT no cabeçalho.
3	Middleware de autenticação valida o JWT; middleware RBAC verifica que o perfil Médico tem permissão de leitura no módulo Prontuário.
4	Controller delega ao ProntuarioService, que chama ProntuarioRepository. PostgreSQL retorna o histórico do paciente.
5	Médico preenche a evolução clínica (queixa, CID-10, conduta) e prescreve medicação no frontend.
6	Frontend envia POST /api/v1/evolucoes com o payload da evolução e os itens de prescrição.
7	ProntuarioService inicia transação de banco de dados: cria Evolucao, cria Prescricao e os Item-Prescricao associados, na mesma transação atômica.
8	Farmacia Service é invocado dentro da mesma transação para verificar a disponibilidade de estoque dos medicamentos prescritos.
9	Transação é confirmada (commit). A prescrição fica disponível imediatamente para o farmacêutico e para o paciente.
10	Backend retorna HTTP 201 Created com os dados da evolução e prescrição. Frontend exibe confirmação ao médico.

13.3 Cenário 2: recepcionista agenda consulta

Este cenário envolve os módulos de Cadastros e Atendimento. Ele exercita a consulta de horários disponíveis, o tratamento de concorrência no agendamento e o envio assíncrono de notificação ao paciente.

Tabela 31 – Cenário arquitetural: agendamento de consulta

#	Passo do Cenário
1	Recepcionista busca paciente por CPF: GET /api/v1/pacientes?cpf=XXX. Sistema retorna o cadastro.
2	Recepcionista seleciona médico, especialidade e data. Frontend solicita horários disponíveis: GET /api/v1/medicos/{id}/disponibilidade?data=YYYY-MM-DD.
3	AgendamentoService consulta a tabela AgendaMedica e filtra horários já ocupados na tabela Consulta. Retorna lista de slots disponíveis.
4	Recepcionista seleciona horário e confirma: POST /api/v1/consultas com os dados do agendamento.

#	Passo do Cenário
5	AgendamentoService tenta inserir a Consulta com o horário selecionado dentro de uma transação com nível de isolamento SERIALIZABLE para prevenir double-booking (condição de corrida).
6	Se o horário foi ocupado simultaneamente por outro usuário, a transação falha com violação de constraint. O backend retorna HTTP 409 Conflict com mensagem descritiva.
7	Se bem-sucedido, a transação é confirmada. O backend dispara assincronamente o envio de e-mail de confirmação via AWS SES, sem bloquear a resposta.
8	Backend retorna HTTP 201 Created. Frontend exibe confirmação à recepcionista em menos de 2 segundos.

13.4 Cenário 3: farmacêutico realiza dispensação com baixa de estoque

Este cenário envolve os módulos de Farmácia, Prontuário e Atendimento. Ele exercita uma transação com múltiplas entidades, garantindo que a dispensação do medicamento, a baixa em estoque e a rastreabilidade da operação sejam registradas de forma consistente.

Tabela 32 – Cenário arquitetural: dispensação de medicamento

#	Passo do Cenário
1	Farmacêutico acessa a prescrição do paciente: GET /api/v1/prescricoes?paciente_id=X. RBAC verifica que o perfil Farmacêutico tem acesso ao módulo Farmácia.
2	Frontend exibe a prescrição com medicamentos, posologia e quantidade disponível em estoque para cada item.
3	Farmacêutico confirma a dispensação: POST /api/v1/dispensacoes com os itens a dispensar.
4	FarmaciaService inicia uma transação: para cada item da prescrição, verifica a quantidade disponível em Medicamento.quantidadeEstoque utilizando SELECT FOR UPDATE para bloquear o registro durante a operação.
5	Para cada item com estoque suficiente, o sistema decrementa Medicamento.quantidadeEstoque e registra a entidade Medicamento_Dispensado, vinculando a dispensação ao farmacêutico, ao medicamento e ao prontuário.
6	Se algum item não tiver estoque suficiente, a transação é revertida integralmente com rollback. O backend retorna HTTP 422 Unprocessable Entity, especificando qual medicamento está em falta.
7	Se bem-sucedida, a transação é confirmada com commit. A prescrição fica registrada como dispensada e disponível para consulta no prontuário.
8	Backend verifica se algum medicamento dispensado atingiu o nível mínimo de estoque. Se sim, registra uma notificação para acompanhamento pelo gestor.
9	Backend retorna HTTP 201 Created com o registro da dispensação. A rastreabilidade é preservada por meio dos identificadores idFarmaceutico, idMedicamento e idProntuario.

13.5 Architecture Models desta Visão (ISO 42010, Seção 5.6)

Tabela 33 – Modelos da visão de cenários

ID	Model Kind	Conteúdo	Versão
MC-01	MK-C1 — Cenário Médico Registra Consulta	10 passos rastreando módulos lógicos, processos e nós físicos (Seção 13.2)	v1.0 — Maio/2026
MC-02	MK-C1 — Cenário Recepcionista Agenda Consulta	8 passos incluindo tratamento de condição de corrida e notificação assíncrona (Seção 13.3)	v1.0 — Maio/2026
MC-03	MK-C1 — Cenário Farmacêutico Dispensação	9 passos com transação ACID, SELECT FOR UPDATE e notificação de estoque mínimo (Seção 13.4)	v1.0 — Maio/2026

13.6 Inconsistências Conhecidas (ISO 42010, Seção 5.7.1)

Não foram identificadas inconsistências conhecidas entre os cenários desta visão e as demais visões arquiteturais.

Os cenários MC-01, MC-02 e MC-03 foram verificados em relação aos módulos definidos na Visão Lógica, aos comportamentos descritos na Visão de Processos, aos artefatos apresentados na Visão de Implementação e aos nós de infraestrutura especificados na Visão Física. Dessa forma, os fluxos descritos permanecem alinhados à estrutura geral da arquitetura do MedClinic.

14 DECISÕES ARQUITETURAIS — ISO/IEC/IEEE 42010

Esta seção documenta as decisões arquiteturais mais significativas do MedClinic, considerando as preocupações arquiteturais, as partes interessadas, as alternativas avaliadas, as justificativas, as consequências e as restrições do projeto. O registro dessas decisões contribui para a rastreabilidade da arquitetura e permite compreender os principais compromissos assumidos durante a definição da solução.

14.1 Rastreabilidade entre preocupações e decisões arquiteturais

A Tabela 34 relaciona as principais preocupações arquiteturais identificadas no projeto às decisões que as endereçam. Essa relação torna explícito o vínculo entre as partes interessadas, suas necessidades e as escolhas arquiteturais adotadas.

Tabela 34 – Rastreabilidade entre preocupações e decisões arquiteturais

Concern (Seção 2.2)	Stakeholders Interessados	Decisão Arquitetural que o endereça
Modificabilidade e separação de responsabilidades	Equipe Dev., Docente, Gestor	DA-001 — Padrão Three-Tier + Cliente-Servidor

Concern (Seção 2.2)	Stakeholders Interessados	Decisão Arquitetural que o endereça
Desempenho e tempo de resposta	Recepcionista, Médico, Enfermeiro, Farmacêutico, Paciente	DA-001 (Three-Tier stateless); DA-002 (JWT sem estado no servidor)
Controle de acesso granular (RBAC / JWT)	Médico, Enfermeiro, Farmacêutico, Gestor, Paciente, Docente	DA-002 — Autenticação stateless com JWT
Interoperabilidade entre Frontend e Backend	Equipe Dev., Docente Orientador	DA-003 — API REST versionada (/api/v1/)
Segurança e privacidade de dados (LGPD)	Paciente, Médico, Gestor, Admin. TI, Docente	DA-004 — Criptografia AES-256-GCM com AWS KMS
Desempenho no upload de arquivos de prontuário	Médico, Paciente, Admin. TI	DA-005 — Upload direto ao S3 via URL pré-assinada
Disponibilidade vs. custo de infraestrutura	Gestor, Admin. TI, Equipe Dev.	DA-006 — Single-AZ no escopo acadêmico; Multi-AZ recomendado para produção
Integridade transacional dos dados clínicos	Médico, Farmacêutico, Gestor, Admin. TI	DA-001 (Three-Tier com transações ACID no serviço); DA-003 (API idempotente)
Conformidade normativa (LGPD, CFM 1.821/2007)	Médico, Gestor, Paciente, Docente, Admin. TI	DA-004 (criptografia); DA-002 (acesso controlado); DA-003 (logs auditoria via API)
Testabilidade e facilidade de deploy	Equipe Dev., Docente Orientador	DA-001 (camadas separadas facilitam mocks); DA-003 (OpenAPI 3.0 p/ testes de contrato)

14.2 Decisões arquiteturais registradas

As decisões a seguir representam escolhas de maior impacto arquitetural no MedClinic. Elas afetam diretamente a estrutura do sistema, a segurança, a integração entre componentes, a infraestrutura e a viabilidade de implantação dentro do escopo acadêmico.

14.2.1 DA-001 — Adoção do padrão Three-Tier com Cliente-Servidor

Tabela 35 – DA-001 — Adoção do padrão Three-Tier com Cliente-Servidor

Campo	Descrição
Concern	Organização estrutural do sistema; modificabilidade; separação de responsabilidades; viabilidade de implementação com equipe júnior em prazo de 14 semanas.

Campo	Descrição
Stakeholders afetados	Equipe de Desenvolvimento — concern: modificabilidade, separação de responsabilidades e trabalho paralelo entre módulos. Docente Orientador — concern: uso correto de padrões arquiteturais e documentação de decisões. Gestor / Admin. de TI — concern: manutenibilidade do sistema após a entrega. Paciente, Recepcionista, Médico, Enfermeiro e Farmacêutico — concern: desempenho e disponibilidade impactados pela escolha estrutural.
Decisão	Adotar o padrão arquitetural Three-Tier (Apresentação / Negócio / Dados), combinado com o padrão Cliente-Servidor, em vez de micros serviços ou arquitetura monolítica sem separação de camadas.
Responsável pela decisão	Equipe de Arquitetura (César, Gabriel L.).
Alternativas consideradas	Microserviços: rejeitado. Overhead de orquestração inviável para a equipe e o prazo estabelecidos. Monólito sem camadas: rejeitado. Baixa manutenibilidade e dificuldade de trabalho paralelo. Three-Tier: escolhido. Modularidade adequada, separação clara de responsabilidades e viabilidade no prazo.
Rationale	O Three-Tier oferece o melhor equilíbrio entre modularidade, necessária para trabalho em equipe; manutenibilidade, necessária para evolução do sistema; e viabilidade, compatível com o prazo e a senioridade da equipe. Atende simultaneamente aos concerns da equipe de desenvolvimento, pela separação de responsabilidades, e dos stakeholders usuários, por desempenho e disponibilidade adequados.
Consequências	Positiva: responsabilidades bem definidas facilitam a divisão de tarefas entre os 6 desenvolvedores. Positiva: mudanças em uma camada têm impacto localizado nas camadas adjacentes. Negativa: overhead de chamadas entre camadas impacta levemente o desempenho.
Restrições / Assumptions	Prazo de 14 semanas; equipe de 6 desenvolvedores júnior; requisito de trabalho paralelo entre Módulo 1 (Backend) e Módulo 2 (Frontend).
Data da decisão	Março de 2026.

14.2.2 DA-002 — Autenticação stateless com JWT

Tabela 36 – DA-002 — Autenticação stateless com JWT

Campo	Descrição
Concern	Segurança de autenticação; controle de acesso granular (RBAC); escalabilidade horizontal; conformidade com RNF-022 da SRS v4.1.

Campo	Descrição
Stakeholders afetados	Todos os usuários internos (Recepcionista, Médico, Enfermeiro, Farmacêutico e Gestor) — concern: acesso seguro e controlado por perfil (RBAC). Paciente — concern: privacidade dos dados; garantia de que apenas ele acessa seus próprios registros. Administrador de TI — concern: escalabilidade horizontal sem armazenamento de estado no servidor. Equipe de Desenvolvimento — concern: simplicidade de implementação e ausência de infraestrutura de cache de sessões.
Decisão	Utilizar JSON Web Tokens (JWT) para gerenciamento de sessões stateless, com expiração de 30 minutos e refresh token de 8 horas, em vez de sessões armazenadas no servidor.
Responsável pela decisão	Equipe de Arquitetura.
Alternativas consideradas	Sessões no servidor: rejeitado. Requer Redis ou banco para estado compartilhado, impedindo escalabilidade horizontal. OAuth 2.0 com provedor externo: rejeitado. Dependência de serviço externo e complexidade desnecessária. JWT stateless: escolhido. Sem estado no servidor, escalável e amplamente suportado.
Rationale	JWT atende simultaneamente ao concern de segurança dos usuários, por meio de autenticação robusta com expiração curta; ao concern de controle de acesso, com perfil embutido no token para RBAC; e ao concern de escalabilidade do Administrador de TI, mantendo o backend stateless. A expiração de 30 minutos minimiza a janela de exposição em caso de vazamento.
Consequências	Positiva: backend completamente stateless; escalabilidade horizontal sem Redis. Positiva: perfil do usuário disponível em cada requisição sem consulta ao banco. Negativa: impossibilidade de invalidação imediata antes da expiração, criando risco residual de segurança.
Restrições / Assumptions	RNF-022 (SRS v4.1) especifica JWT; suporte a 50 usuários simultâneos sem degradação.
Data da decisão	Março de 2026.

14.2.3 DA-003 — API REST versionada como interface entre Frontend e Backend

Tabela 37 – DA-003 — API REST versionada como interface entre Frontend e Backend

Campo	Descrição
Concern	Desacoplamento entre camadas; modificabilidade; testabilidade; trabalho paralelo entre equipes de frontend e backend.

Campo	Descrição
Stakeholders afetados	Equipe de Desenvolvimento — concern: desenvolvimento paralelo dos Módulos 1 e 2; testabilidade com Postman/Swagger; modificabilidade independente de cada camada. Docente Orientador — concern: conformidade com boas práticas de design de APIs, como OpenAPI 3.0 e versionamento. Administrador de TI — concern: manutenibilidade e capacidade de atualizar o backend sem quebrar o frontend. Todos os usuários finais — concern: disponibilidade e desempenho impactados pelo overhead de serialização REST/JSON.
Decisão	Toda comunicação entre frontend e backend ocorre exclusivamente por API RESTful com versionamento (/api/v1/), utilizando JSON como formato de troca de dados e verbos HTTP semânticos.
Responsável pela decisão	Equipe de Arquitetura.
Alternativas consideradas	GraphQL: considerado. Flexível, mas com curva de aprendizado alta para equipe júnior. gRPC: rejeitado. Incompatível com browsers sem proxy. REST versionado: escolhido. Padrão amplamente conhecido, facilmente testável e adequado ao escopo do projeto.
Rationale	REST atende ao concern de desacoplamento, permitindo que frontend e backend evoluam independentemente; ao concern de testabilidade, por meio de documentação OpenAPI 3.0; e ao concern de modificabilidade, pois o versionamento garante retrocompatibilidade. É a opção que melhor equilibra os concerns conflitantes entre a equipe de desenvolvimento e os usuários finais.
Consequências	Positiva: frontend e backend podem ser testados e desenvolvidos independentemente. Positiva: API documentada em OpenAPI 3.0 facilita testes de contrato e onboarding. Negativa: overhead de serialização/desserialização JSON impacta ligeiramente o desempenho.
Restrições / Assumptions	RNF-019 (OpenAPI 3.0); RNF-023 (versionamento de API); desenvolvimento paralelo entre M1 e M2.
Data da decisão	Março de 2026.

14.2.4 DA-004 — Criptografia AES-256-GCM com chaves gerenciadas pelo AWS KMS

Tabela 38 – DA-004 — Criptografia AES-256-GCM com chaves gerenciadas pelo AWS KMS

Campo	Descrição
Concern	Proteção de dados sensíveis em repouso; conformidade com LGPD, Art. 46, e Resolução CFM nº 1.821/2007; privacidade dos dados de saúde.

Campo	Descrição
Stakeholders afetados	Paciente — concern: privacidade dos dados de saúde; garantia de que CPF e informações clínicas não ficam expostos em texto claro. Médico e Enfermeiro — concern: conformidade normativa com o CFM e proteção de prontuários eletrônicos. Gestor e Admin. de TI — concern: conformidade com LGPD e auditabilidade de acesso às chaves via CloudTrail. Equipe de Desenvolvimento — concern: gerenciamento seguro de chaves sem hardcoding. Docente Orientador — concern: conformidade com requisitos de segurança especificados na SRS v4.1.
Decisão	Criptografar colunas de dados sensíveis, como CPF e dados clínicos, com AES-256-GCM, utilizando chaves gerenciadas pelo AWS Key Management Service (KMS), com rotação automática anual.
Responsável pela decisão	Equipe de Segurança (Rafael, Orlando).
Alternativas consideradas	RDS encryption at rest apenas: insuficiente isoladamente, pois protege o armazenamento, mas não impede a exposição de dados já descritografados pela aplicação ou por acessos indevidos ao banco. Chaves hardcoded: rejeitado. Viola RNF-007 e expõe dados a todos com acesso ao repositório. AES-256-GCM + AWS KMS: escolhido. Gerenciamento centralizado, rotação automática e auditoria via CloudTrail.
Rationale	Esta decisão endereça diretamente o concern de privacidade do Paciente e o concern de conformidade do Médico e do Gestor. O AWS KMS permite auditoria de cada uso de chave, enquanto o AES-256-GCM garante integridade além da confidencialidade. A rotação automática reduz o risco de comprometimento prolongado.
Consequências	Positiva: conformidade com LGPD e CFM. Positiva: auditoria completa de uso de chaves via CloudTrail. Negativa: latência adicional por chamadas ao KMS. Negativa: dependência da disponibilidade do AWS KMS para acessar dados criptografados.
Restrições / Assumptions	RNF-005 (SRS v4.1); LGPD, Art. 46; ambiente de hospedagem exclusivamente AWS.
Data da decisão	Abril de 2026.

14.2.5 DA-005 — Upload direto de arquivos ao S3 via URL pré-assinada

Tabela 39 – DA-005 — Upload direto de arquivos ao S3 via URL pré-assinada

Campo	Descrição
Concern	Desempenho no upload de arquivos de prontuário; disponibilidade do servidor de aplicação para usuários concorrentes; segurança no acesso temporário a arquivos.
Stakeholders afetados	Médico — concern: upload rápido ao prontuário sem degradar o sistema. Paciente — concern: acesso seguro e temporário a seus próprios documentos, como resultados e atestados. Recepcionista e demais usuários — concern: desempenho do sistema não degradado durante uploads simultâneos. Admin. de TI — concern: redução de carga no EC2 t3.micro. Equipe de Desenvolvimento — concern: complexidade de implementação do two-step upload.
Decisão	O backend gera URLs pré-assinadas do AWS S3 com validade de 15 minutos, e o upload do arquivo é realizado diretamente do navegador ao S3, sem trafegar pelo EC2.
Responsável pela decisão	Equipe de Infraestrutura (César, Lucas).
Alternativas consideradas	Upload via backend: rejeitado. Consome memória do EC2 e degrada desempenho para outros usuários. URL pré-assinada S3: escolhida. Upload direto; o EC2 gerencia apenas metadados e a URL.
Rationale	Esta decisão resolve o conflito entre o concern de desempenho do Médico, que precisa de upload rápido, e o concern de disponibilidade dos demais usuários, pois o EC2 não fica congestionado. A validade limitada de 15 minutos endereça o concern de segurança do Paciente, evitando URL reutilizável indefinidamente. O custo é a maior complexidade de implementação para a equipe de desenvolvimento.
Consequências	Positiva: EC2 descarregado de tráfego de arquivos. Positiva: uploads paralelos sem competição por recursos do backend. Negativa: implementação de two-step upload adiciona complexidade para a equipe de desenvolvimento. Negativa: inconsistência potencial de metadado se o upload falhar após geração da URL.
Restrições / Assumptions	RI-009 (SRS v4.1); limite de memória do EC2 t3.micro; RNF-001 (tempo de resposta $\leq 2s$).
Data da decisão	Abril de 2026.

14.2.6 DA-006 — Single-AZ para RDS no escopo acadêmico

Tabela 40 – DA-006 — Single-AZ para RDS no escopo acadêmico

Campo	Descrição
Concern	Disponibilidade do banco de dados versus custo de infraestrutura; restrição orçamentária do projeto acadêmico.
Stakeholders afetados	Gestor — concern: custo de infraestrutura dentro do orçamento de R\$ 150,00/mês e disponibilidade do sistema para operação da clínica. Recepcionista, Médico, Enfermeiro e Farmacêutico — concern: disponibilidade do sistema; indisponibilidade impacta atendimentos. Paciente — concern: disponibilidade para agendamentos e acesso a documentos. Admin. de TI — concern: estratégia de backup e recuperação; tempo de retorno em caso de falha ($MTTR \leq 15$ min). Equipe de Desenvolvimento e Docente Orientador — concern: custo compatível com o orçamento acadêmico e documentação transparente da limitação.
Decisão	Utilizar instância RDS Single-AZ (db.t3.micro) no ambiente acadêmico, com documentação explícita da limitação e recomendação de migração para Multi-AZ antes de qualquer deploy em produção real.
Responsável pela decisão	Equipe de Infraestrutura.
Alternativas consideradas	Multi-AZ RDS: SLA de 99,95% e failover automático. Custo aproximadamente 2x maior, inviável no orçamento acadêmico. Single-AZ: escolhido. Sem SLA formal por instância individual, porém com custo dentro do orçamento.
Rationale	Esta decisão representa o tradeoff mais explícito do projeto entre o concern de disponibilidade dos usuários operacionais, como Recepcionista, Médico e Farmacêutico, e o concern de custo do Gestor. Para o contexto acadêmico, a meta de 95% de disponibilidade é aceitável. Para um deploy real, a indisponibilidade impacta diretamente atendimentos médicos, tornando Multi-AZ obrigatório. A decisão é documentada com transparência para atender ao concern do Docente Orientador.
Consequências	Positiva: custo dentro do orçamento acadêmico. Negativa: sem failover automático, podendo ocorrer indisponibilidade durante manutenção do RDS. Negativa: risco de perda de disponibilidade por falha de hardware, concern crítico para Médico e Farmacêutico. Decisão futura: migração para Multi-AZ deve ser a primeira ação antes de qualquer deploy em produção real.
Restrições / Assumptions	Orçamento de R\$ 150,00/mês para cloud; prazo acadêmico até julho de 2026; RNF-010 revisado (SRS v4.1).
Data da decisão	Abril de 2026.

REFERÊNCIAS

KRUCHTEN, Philippe. Architectural Blueprints — The “4+1” View Model of Software Architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, nov. 1995.

ISO/IEC/IEEE 42010:2011. *Systems and software engineering — Architecture description*. International Organization for Standardization, 2011.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. 3. ed. Addison-Wesley, 2012.

Documento de Requisitos de Software — MedClinic SRS v4.1. Departamento de Computação — UFS, Maio de 2026.

Descrição de Casos de Uso — MedClinic v3.0. Departamento de Computação — UFS, Maio de 2026.

Proposta de Plano de Projeto — Sistema de Gestão para Clínica Médica. Departamento de Computação — UFS, Abril de 2026.

Tabelas do Mapeamento Objeto-Relacional — MedClinic. Departamento de Computação — UFS, 2026.